
Sprawozdanie z Laboratorium: Bazy Danych

Michał Bednarczyk

Jul 01, 2026

SPIS TREŚCI:

1	Wstęp	1
1.1	Wstęp do sprawozdania	1
1.2	Wnioski z przeprowadzonych ćwiczeń	1
1.2.1	1. Główne repozytorium sprawozdania	1
1.2.2	2. Repozytorium z modelami fizycznymi naszej bazy	1
1.2.3	3. Repozytoria reszty grupy	1
2	Badania literaturowe	2
2.1	Sprzęt dla bazy danych	2
2.1.1	Wstęp	2
2.1.2	Procesor (CPU)	2
2.1.3	Pamięć RAM	3
2.1.4	Pamięć Masowa	4
2.1.5	Kontrolery i Interfejsy Pamięci	5
2.1.6	Zasilanie i Niezawodność	5
2.1.7	Podsumowanie	6
2.2	Konfiguracja bazy danych PostgreSQL	6
2.2.1	Wstęp	6
2.2.2	Podstawowe parametry	6
2.2.3	Lokalizacja i struktura katalogów PostgreSQL	6
2.2.4	Środowiska i profile	7
2.2.5	Automatyzacja	7
2.2.6	Przykłady	7
2.2.7	Najlepsze praktyki	8
2.2.8	Podsumowanie	8
2.2.9	Szczegółowe omówienie plików konfiguracyjnych	8
2.2.10	postgresql.conf	8
2.2.11	pg_hba.conf	11
2.2.12	pg_ident.conf	12
2.3	Kontrola i konserwacja	13
2.3.1	Wprowadzenie i planowanie konserwacji	13
2.3.2	Zarządzanie stanem serwera i sesjami	14
2.3.3	Proces Vacuum	14
2.3.4	Zarządzanie transakcjami i schematy (SCHEMA)	15
2.3.5	Zarządzanie indeksami	15
2.3.6	Podsumowanie	15
2.4	Monitorowanie i diagnostyka	15
2.4.1	Wstęp	16
2.4.2	Sesje i użytkownicy	16
2.4.3	Śledzenie dostępu użytkowników do tabel	16
2.4.4	Błędy dziennika, logi i raporty	17
2.4.5	Monitorowanie na poziomie systemu operacyjnego	17

2.4.6	Monitorowanie serwera	17
2.4.7	Znaczenie monitorowania i diagnostyki w bazach danych	18
2.4.8	Podsumowanie	18
2.4.9	Źródła	18
2.5	Wydajność, skalowanie i replikacja	18
2.5.1	Wydajność, skalowanie i replikacja	18
2.5.2	Kontrola i buforowanie połączeń z bazą danych	18
2.5.3	INDEX i CLUSTER	19
2.5.4	Rola i zastosowanie replikacji	19
2.5.5	Oprogramowanie i zaimplementowane mechanizmy replikacji	19
2.5.6	Limity systemu oraz ograniczanie dostępu użytkowników	20
2.5.7	Testy wydajności sprzętu na poziomie systemu operacyjnego	20
2.6	Partycjonowanie danych	20
2.6.1	Wstęp i istota problemu	20
2.6.2	Cele stosowania partycjonowania danych	21
2.6.3	Zastosowanie partycjonowania	21
2.6.4	Zalety i wady partycjonowania	21
2.6.5	Implementacja partycjonowania w PostgreSQL	22
2.6.6	Ograniczenia i obejście w środowisku SQLite	22
2.7	Bezpieczeństwo Baz Danych	22
2.7.1	Model bezpieczeństwa CIA i ochrona infrastruktury	22
2.7.2	Ataki na bazy danych i luki w zabezpieczeniach	23
2.7.3	Zarządzanie tożsamością, uwierzytelnianie i autoryzacja	25
2.7.4	Kryptografia w bazach danych	27
2.7.5	Audyt, logowanie i wykrywanie anomalii	28
2.7.6	Bezpieczeństwo kopii zapasowych i Disaster Recovery	29
2.8	Kopie zapasowe i odzyskiwanie danych	30
2.8.1	Wstęp	30
2.8.2	Tworzenie kopii zapasowej całego systemu PostgreSQL - mechanizmy wbudowane	30
2.8.3	Tworzenie kopii zapasowych poszczególnych baz danych - mechanizmy wbudowane	31
2.8.4	Odzyskiwanie usuniętych lub uszkodzonych danych	31
2.8.5	Dedykowane oprogramowanie i skrypty zewnętrzne do automatyzacji	32
2.8.6	Podsumowanie	33
3	Planowanie baz danych i tworzenie dokumentacji	34
3.1	Wybór zagadnienia i identyfikacja danych	34
3.2	Opis procesów i towarzyszących im danych	34
3.3	Prototypowy plik JSON	34
3.3.1	Dobór typów danych	35
3.4	Modelowanie koncepcyjne i logiczne	35
3.4.1	Model koncepcyjny	35
3.4.2	Model koncepcyjny – notacja Chena	36
3.4.3	Model logiczny i normalizacja	36
3.4.4	Schemat notacji IE po normalizacji	36
4	Definiowanie bazy danych i wprowadzanie danych do bazy	37
4.1	Definicja bazy danych w wariantach PostgreSQL i SQLite	37
4.2	Wybór mechanizmu wprowadzania danych	39
4.3	Komentarz do procesu wprowadzania danych	39
5	Zapytania do bazy danych	40
5.1	Scieżki do wykonanych zapytań na naszej bazie:	40
5.2	Funkcje zapytań dla PostgreSQL	40
5.3	Funkcje zapytań dla SQLITE	41

Autor

Michał Bednarczyk

1.1 Wstęp do sprawozdania

Głównym tematem zajęć laboratoryjnych było praktyczne przejście przez pełen cykl życia projektu relacyjnej bazy danych dla sklepu elektronicznego. Ćwiczenia rozpoczęły się od stworzenia kompletnej dokumentacji projektowej, obejmującej analizę procesów, prototypowanie w formacie JSON oraz tworzenie modeli koncepcyjnych i logicznych (z użyciem diagramów Chena i notacji IE). Zwieńczeniem tej pracy było przełożenie przygotowanych schematów na rzeczywisty model fizyczny w środowiskach PostgreSQL i SQLite, zasilenie ich danymi oraz opracowanie zapytań.

1.2 Wnioski z przeprowadzonych ćwiczeń

Zrealizowanie całego procesu na laboratoriach udowodniło, że rzetelnie stworzona dokumentacja i poprawna normalizacja są absolutnym fundamentem do bezbłędnego wdrożenia modelu fizycznego. Podczas implementacji i testowania obu środowisk wyraźnie zarysowała się przewaga PostgreSQL nad SQLite w kontekście budowania profesjonalnych systemów. Rygorystyczne typowanie danych, bogatsza składnia oraz natywne, ścisłe egzekwowanie więzów integralności w PostgreSQL sprawiają, że jest to rozwiązanie znacznie bezpieczniejsze i bardziej przewidywalne niż uproszczony, plikowy SQLite. Etap końcowy, polegający na tworzeniu skomplikowanych zapytań (z wielokrotnymi złączeniami i podzapytaniami), ostatecznie potwierdził, że choć SQLite jest wygodny w prostych zastosowaniach, to właśnie PostgreSQL oferuje stabilność, precyzję i wydajność niezbędną do profesjonalnego przetwarzania złożonych informacji.

1.2.1 1. Główne repozytorium sprawozdania

Link do mojego repozytorium ze sprawozdaniem:

- **Link:** <https://github.com/Koko9077/Sprawozdanie>

1.2.2 2. Repozytorium z modelami fizycznymi naszej bazy

- **Link:** <https://github.com/Koko9077/Model-fizyczny-baz>

1.2.3 3. Repozytoria reszty grupy

Poniżej znajdują się odnośniki do repozytoriów reszty grupy:

- **Grupa 1 (rozdział_1):** <https://github.com/karaskamil/Sprzet-dla-bazy-danych.git>
- **Grupa 2 (rozdział_2):** https://github.com/Youarecheck/Bazy_Danych_Tematyczne_Repo_MK.git
- **Grupa 3 (rozdział_3):** <https://github.com/pawlos1337/Bazy-danych-temat.git>
- **Grupa 4 (rozdział_4):** https://github.com/OskarProgrammer/monitorowanie_i_diagnostyka.git
- **Grupa 5 (rozdział_5):** <https://github.com/KMachoK/Tematyczne.git>
- **Grupa 6 (rozdział_6):** <https://github.com/domino0472/Partycjonowani-Danych>
- **Grupa 7 (rozdział_7):** <https://github.com/oski486/BazyDanych-Subject.git>
- **Grupa 8 (rozdział_8):** <https://github.com/Koko9077/Kopie-zapasowe-i-odzyskiwanie-danych.git>

BADANIA LITERATUROWE

2.1 Sprzęt dla bazy danych

Autorzy

1. Wiktor Głogowski
2. Olga Grześkowiak
3. Kamil Karaś

2.1.1 Wstęp

Współczesne systemy zarządzania bazami danych (DBMS) stanowią fundament operacyjny większości przedsiębiorstw. Efektywność ich działania zależy bezpośrednio od synergii pomiędzy warstwą oprogramowania a architekturą sprzętową, na której są one uruchamiane. Bazy danych wykazują unikalne, skrajnie wysokie wymagania względem podsystemów wejścia/wyjścia (I/O), procesora, pamięci operacyjnej oraz niezawodności zasilania.

W zależności od profilu obciążenia systemu, infrastrukturę sprzętową klasyfikuje się pod dwa główne typy zadań:

- **OLTP (Online Transaction Processing):** Systemy transakcyjne charakteryzujące się ogromną liczbą jednoczesnych, krótkich operacji zapisu i odczytu (np. systemy bankowe, sklepy internetowe). Kluczowym parametrem jest tu minimalne opóźnienie oraz wysoka liczba operacji wejścia/wyjścia na sekundę (IOPS).
- **OLAP (Online Analytical Processing):** Hurtownie danych i systemy analityczne realizujące długie, złożone zapytania, które agregują miliony rekordów. Tutaj priorytetem jest przepustowość sekwencyjna oraz masowa wielordzeniowość procesorów.

Poniższy rozdział opisuje architekturę fizyczną bazy danych oraz wytyczne w sprawie doboru komponentów.

2.1.2 Procesor (CPU)

Wybór procesora dla serwera bazodanowego determinowany jest nie tylko surową wydajnością obliczeniową, ale również architekturą pamięci podręcznej, topologią połączeń międzygniazdowych oraz modelami licencjonowania oprogramowania.

Wydajność Jednowątkowa vs Wielordzeniowość

Decyzja o wyborze modelu procesora zależy od struktury zapytań generowanych przez aplikację:

1. **Taktowanie Zegara:** W środowiskach OLTP, gdzie transakcje często wykonują się sekwencyjnie lub są blokowane przez zamki i zatraski, wyższe taktowanie rdzenia (np. 3.6 GHz - 4.0 GHz) przynosi lepsze rezultaty niż ekstremalna liczba rdzeni o niskim taktowaniu. Szybka realizacja pojedynczego wątku skraca czas trwania blokad.
2. **Skalowanie Wielordzeniowe:** W systemach OLAP silniki baz danych potrafią dzielić jedno zapytanie na wiele wątków. W tym przypadku procesory posiadające 64, 96 lub więcej rdzeni fizycznych (np. AMD EPYC, Intel Xeon Scalable) pozwalają na równoległe skanowanie indeksów i tabel, drastycznie przyspieszając operacje analityczne.

Wpływ Licencjonowania na Wybór CPU: Wiodący producenci oprogramowania bazodanowego (np. Microsoft SQL Server Enterprise, Oracle) stosują model licencjonowania per-core. Koszt licencji na jeden rdzeń wielokrotnie przewyższa koszt zakupu samego sprzętu. Z tego powodu ekonomicznie i wydajnościowo uzasadniony jest wybór procesorów o mniejszej liczbie rdzeni, ale o maksymalnym dostępnym taktowaniu bazowym i zaawansowanej architekturze cache.

Topologia NUMA (Non-Uniform Memory Access)

Wieloprocessorowe serwery (np. architektury dwu- lub czterogniazdowe) dzielą zasoby na tzw. węzły NUMA. Każdy procesor posiada swój zintegrowany kontroler pamięci i bezpośredni dostęp do fizycznie najbliższych banków RAM (pamięć lokalna).

- **Dostęp Lokalny:** Charakteryzuje się minimalnymi opóźnieniami i maksymalnym pasmem przenoszenia.
- **Dostęp Zdalny:** Jeśli procesor osadzony w gnieździe 0 potrzebuje danych znajdujących się w banku pamięci przypisanym do gniazda 1, transakcja musi odbyć się za pośrednictwem magistrali międzyprocesorowej (Intel UPI lub AMD Infinity Fabric). Generuje to narzut opóźnień zwany *NUMA penalty*.

Niewłaściwa alokacja wątków przez system operacyjny może prowadzić do ciągłego przemieszczania danych między węzłami (*NUMA bouncing*), co drastycznie obniża wydajność bazy danych. Wymagane jest stosowanie silników bazodanowych w pełni wspierających i optymalizujących architekturę NUMA (*NUMA-aware*).

Znaczenie Pamięci Podręcznej (Cache L3)

Struktury danych w relacyjnych bazach (drzewa B+, tabele stron) sprawiają, że procesor wykonuje ogromną liczbę operacji skoków i wyszukiwań wskaźników. Powoduje to częste chybień pamięci podręcznej, zmuszając procesor do czekania na dane z RAM-u. Procesory wyposażone w powiększoną pamięć podręczną poziomu trzeciego (L3), takie jak układy z technologią 3D V-Cache, pozwalają na zatrzymanie krytycznych struktur indeksowych wewnątrz struktur procesora, generując ogromne zyski wydajnościowe.

2.1.3 Pamięć RAM

Pamięć operacyjna RAM decyduje o tym, jak duża część bazy danych może być przetwarzana bezpośrednio w super-szybkiej pamięci półprzewodnikowej, bez konieczności odwoływania się do podsystemu dyskowego.

Mechanizm Buforowania Danych

Podstawową jednostką wymiany danych między dyskiem a pamięcią w bazach danych jest strona (zazwyczaj o rozmiarze 8 KB). Silnik bazy danych alokuje obszar pamięci RAM nazywany **Buffer Pool** (w MS SQL Server) lub **Shared Buffers** (w PostgreSQL).

- **Odczyty z Pamięci (Logical Reads):** Jeśli żądana strona danych znajduje się już w RAM, odczyt następuje natychmiastowo (rzędu nanosekund).
- **Odczyty z Dysku (Physical Reads):** W przypadku braku strony w buforze, wątek bazy danych zostaje zawieszony do czasu fizycznego załadowania strony z pamięci masowej, co zwiększa czas odpowiedzi o rzędy wielkości.

Docelowym stanem konfiguracyjnym dla systemów produkcyjnych jest posiadanie takiej ilości pamięci RAM, która pozwala na bezproblemowe przetrzymywanie w całości tzw. hot data set, czyli najczęściej modyfikowanych i odczytywanych danych wraz z ich indeksami.

Wymóg Technologii ECC (Error-Correcting Code)

W serwerach bazodanowych niedopuszczalne jest stosowanie pamięci bez obsługi korekcji błędów (non-ECC). Pamięci ECC posiadają dodatkowe układy scalone pozwalające na wykrywanie i automatyczną korekcję błędów jednobitowych oraz wykrywanie błędów wielobitowych.

Wystąpienie zjawiska *bit-flip* (samorzutnej zmiany wartości bitu w pamięci RAM pod wpływem np. tła radiacyjnego lub zakłóceń elektromagnetycznych) w systemie bez ECC niesie katastrofalne skutki dla baz danych:

- **Cicha korupcja danych:** Błędna wartość może zostać zapisana z bufora pamięci wprost na dysk, powodując trwałe i niezauważalne zniszczenie struktury tabeli lub błędne zapisy księgowe.
- **Uszkodzenie stron indeksowych:** Powoduje błędy integralności i uniemożliwia poprawne wykonywanie zapytań SQL.
- **Awaria krytyczna:** W przypadku naruszenia pamięci jądra systemu operacyjnego lub krytycznego procesu bazy danych dochodzi do natychmiastowego zatrzymania serwera (Kernel Panic / BSOD).

Parametry Wydajnościowe: Przepustowość i Opóźnienia

Współczesne standardy serwerowe opierają się na pamięciach DDR5, oferujących zaawansowaną architekturę wielokanałową (np. 8 lub 12 kanałów pamięci na jedno gniazdo CPU). Wysoka przepustowość (wyrażana w MT/s) przyspiesza operacje sortowania w pamięci, tworzenie tabel tymczasowych oraz operacje złączania typu *Hash Join*, które są na porządku dziennym w systemach analitycznych.

2.1.4 Pamięć Masowa

Podsystem pamięci masowej bezpośrednio determinuje ostateczną trwałość danych (cecha *Durability* z paradygmatu ACID) oraz jest najczęstszym wąskim gardłem systemu informatycznego z uwagi na mechaniczne lub interfejsowe ograniczenia prędkości.

Klasyfikacja Nośników i Ich Przydatność

Współczesna inżynieria bazodanowa całkowicie odchodzi od talerzowych dysków magnetycznych HDD w środowiskach produkcyjnych na rzecz pamięci półprzewodnikowych Flash.

- **Dyski HDD (SAS 10K/15K):** Ze względu na ograniczenia mechaniczne oferują zaledwie kilkaset IOPS i wysokie opóźnienia (>5 ms). Ich zastosowanie ogranicza się obecnie wyłącznie do składowania archiwalnych kopii zapasowych (backups).
- **Dyski SSD (SATA/SAS Enterprise):** Zapewniają stabilną wydajność na poziomie do 90 000 IOPS dla odczytu, jednak ograniczeniem staje się interfejs SATA (6 Gb/s) lub SAS-3 (12 Gb/s).
- **Nośniki NVMe (PCIe Gen4/Gen5):** Komunikują się bezpośrednio z procesorem poprzez magistralę PCI Express, eliminując narzut protokołów SCSI/ATA. Osiągają wydajność przekraczającą 1 000 000 IOPS oraz redukują opóźnienia do wartości mikrosekundowych (<50 µs), stając się bezdyskusyjnym standardem dla baz danych.

Fizyczna Separacja Architektury Plików

Poprawna konfiguracja pamięci masowej wymaga rozdzielenia różnych typów aktywności wejścia/wyjścia na odseparowane fizycznie wolumeny dyskowe:

1. **Dziennik Transakcji:** Każda operacja modyfikacji danych (INSERT, UPDATE, DELETE) musi zostać najpierw sekwencyjnie zapisana w dzienniku transakcji, zanim baza potwierdzi jej pomyślne wykonanie. Ruch ten ma charakter niemal wyłącznie sekwencyjnego zapisu o krytycznym znaczeniu dla opóźnienia aplikacji. Dziennik transakcji powinien znajdować się na najszybszych dedykowanych nośnikach o minimalnym opóźnieniu zapisu.
2. **Pliki Danych:** Przechowują właściwe tabele i indeksy. Ruch na tych plikach ma charakter wysoce losowy (zarówno odczyty, jak i zapisy wywoływane przez proces *Checkpoint*, który zrzuca zmodyfikowane strony z RAM na dysk). Wymagają macierzy zoptymalizowanej pod kątem wysokiej liczby losowych IOPS.
3. **Baza Tymczasowa (TempDB / Swapping Space):** Wykorzystywana przez silnik bazy do przechowywania obiektów tymczasowych, wyników pośrednich podzapytań oraz sortowania. Generuje bardzo intensywny, mieszany ruch. Umieszcza się ją na ultra-szybkich dyskach o wysokiej wytrzymałości.

Żywotność i Wytrzymałość Zapisu (DWPD)

Bazy danych nieustannie modyfikują i nadpisują sektory. Z tego powodu stosowanie dysków klasy konsumenckiej grozi ich błyskawicznym zużyciem. Przy doborze nośników kluczowym parametrem jest **DWPD** (*Drive Writes Per Day*) – wskaźnik określający, ile razy dziennie można zapisać całą pojemność dysku przez okres gwarancyjny (zazwyczaj 5 lat). Do baz danych wymagane są nośniki klasy *Enterprise Mixed-Use* lub *Write-Intensive* o współczynniku DWPD wynoszącym minimum 3-5.

2.1.5 Kontrolery i Interfejsy Pamięci

Kontrolery realizują zadania pośrednictwa, zarządzania macierzami nadmiarowymi (RAID) oraz gwarantują integralność zapisu.

Sprzętowy RAID vs Software-Defined Storage

Przez lata standardem były sprzętowe kontrolery RAID (*Hardware RAID*) wyposażone we własny procesor i pamięć cache. Odciażają one procesor główny serwera z obliczeń sum kontrolnych (szczególnie parzystości w RAID 5/6). W przypadku nowoczesnych pamięci NVMe sprzętowe kontrolery starszego typu stawały się wąskim gardłem, blokując bezpośredni dostęp do linii PCIe. Współcześnie odchodzi się od nich na rzecz:

- **Bezpośredniego podłączenia NVMe (Direct Attached PCIe)** i zarządzania programowego na poziomie zaawansowanych systemów plików (np. ZFS) lub warstw logicznych OS (np. Linux mdadm / LVM).
- Modernistycznych, dedykowanych sprzętowych kontrolerów NVMe RAID klasy Enterprise, zdolnych do natywnego przetwarzania protokołu NVMe z pełną prędkością magistrali PCIe Gen5.

Pamięć Podtrzymywana Bateriajnie (BBU/FBWC): W przypadku korzystania ze sprzętowego kontrolera RAID z włączoną pamięcią podręczną zapisu (*Write-Back Cache*), **bezwzględnym warunkiem bezpieczeństwa** jest obecność modułu **BBU** (*Battery Backup Unit*) lub **FBWC** (*Flash-Backed Write Cache*). W razie awarii zasilania serwera, moduł ten podtrzymuje dane w pamięci cache kontrolera (lub przepisuje je do dedykowanej pamięci flash), zapobiegając utracie nieutralizowanych transakcji i uszkodzeniu struktury macierzy.

Wybór Poziomu RAID pod Kątem Baz Danych

- **RAID 10 (Striped Mirror):** Rekomendowany układ dla większości systemów bazodanowych. Łączy zalety wydajnościowe paskowania (RAID 0) i bezpieczeństwa lustrzanego (RAID 1). Zapewnia doskonałą wydajność losowego zapisu, ponieważ nie generuje tzw. narzutu parzystości (*write penalty*).
- **RAID 5 / RAID 6:** Zapewniają lepsze wykorzystanie przestrzeni dyskowej, ale każda operacja zapisu wymaga odczytu danych, obliczenia nowej parzystości i ponownego zapisu (*kara za zapis*). Są **wysoce niewskazane dla baz transakcyjnych OLTP**. Mogą być stosowane jedynie w hurtowniach danych (OLAP), gdzie operacje zapisu są sekwencyjne i rzadkie (procesy ETL).

2.1.6 Zasilanie i Niezawodność

Podsystem zasilania w infrastrukturze bazodanowej traktowany jest jako integralny element ochrony integralności logicznej danych, a nie tylko komponent ciągłości działania.

Nadmiarowość Zasilaczy i Niezależne Linie Zasilające

Serwer bazodanowy musi posiadać zasilacze nadmiarowe typu *Hot-Swap* (z możliwością wymiany podczas pracy) w konfiguracji minimum **N+1** lub optymalnie **2N** (Full Redundancy).

Zasilacze muszą być podpięte do fizycznie odseparowanych źródeł zasilania:

- **Zasilacz A:** Podłączony do Linii zasilającej A oraz dedykowanego modułu UPS A.
- **Zasilacz B:** Podłączony do Linii zasilającej B oraz dedykowanego modułu UPS B.

Taka topologia gwarantuje, że awaria jednego zasilacza, uszkodzenie kabla lub całkowity zanik napięcia na jednej z linii energetycznych nie spowoduje wyłączenia serwera.

Infrastruktura UPS i Agregaty Prądotwórcze

Wszelkie systemy serwerowe DBMS muszą współpracować z zasilaczami awaryjnymi UPS działającymi w topologii *Online* (podwójne przetwarzanie). Zapewniają one idealną sinusoidę napięcia, eliminują przepięcia i gwarantują czas podtrzymania niezbędny do automatycznego uruchomienia spalinowych agregatów prądotwórczych o odpowiedniej mocy.

Zapobieganie Zjawisku “Torn Writes”

Zjawisko *Torn Write* (rozbity/częściowy zapis) występuje, gdy system operacyjny przesyła do zapisu stronę danych (np. 8 KB), a w trakcie fizycznego zapisu na komórkę pamięci Flash dochodzi do nagłego odcięcia zasilania. W efekcie na dysku ląduje np. tylko 4 KB danych, co powoduje bezpowrotne uszkodzenie struktury tej strony i błąd sumy kontrolnej przy próbie kolejnego uruchomienia bazy danych.

Poza mechanizmami programowymi (takimi jak *Doublewrite Buffer* w silniku InnoDB/MySQL), na poziomie sprzętowym stosuje się technologię **PLP (Power Loss Protection)**. Dyski klasy Enterprise posiadają wbudowane banki kondensatorów tantalowych. W momencie wykrycia spadku napięcia, energia z kondensatorów pozwala na podtrzymanie pracy kontrolera dysku przez czas wymagany do bezpiecznego opróżnienia pamięci podręcznej RAM dysku i utrwalenia wszystkich danych w nieulotnych komórkach pamięci NAND Flash.

2.1.7 Podsumowanie

Dobór sprzętu dla systemów zarządzania bazami danych musi być ściśle dopasowany do profilu obciążenia, z wyraźnym podziałem na transakcyjne środowiska OLTP oraz analityczne systemy OLAP. Wybór procesora stanowi kompromis pomiędzy taktowaniem a wielordzeniowością ze względu na koszty licencyjne, podczas gdy w przypadku pamięci operacyjnej absolutnym wymogiem jest zastosowanie technologii ECC chroniącej przed cichą korupcją danych. W obszarze pamięci masowej standardem stały się nośniki półprzewodnikowe NVMe o wysokiej wytrzymałości (DWPD), które ze względów wydajnościowych wymagają fizycznej separacji wolumenów dla dzienników transakcji i plików danych. Do zabezpieczenia pracy dysków rekomenduje się użycie konfiguracji RAID 10 w połączeniu z nowoczesnymi kontrolerami sprzętowymi lub rozwiązaniami programowymi. Całość infrastruktury musi być kategorycznie chroniona przed awariami poprzez nadmiarowe zasilacze w konfiguracji 2N, systemy UPS w topologii Online oraz dyski wyposażone w sprzętową ochronę PLP eliminującą zjawisko uszkodzonych zapisów.

2.2 Konfiguracja bazy danych PostgreSQL

2.2.1 Wstęp

Rozdział opisuje konfigurację połączenia z bazą danych **PostgreSQL** – dojrzałym, open-source’owym systemem zarządzania relacyjnymi bazami danych (RDBMS), który wyróżnia się pełną zgodnością ze standardem SQL oraz silnym naciskiem na rozszerzalność i niezawodność.

Wymagania:

- dostęp do instancji PostgreSQL,
- zmienne środowiskowe,
- narzędzia projektowe (np. `psql`, `pg_isready`).

2.2.2 Podstawowe parametry

Każde połączenie z PostgreSQL wymaga:

- hosta i portu (domyślnie `localhost` i `5432`),
- nazwy bazy danych,
- użytkownika i hasła.

Dane wrażliwe (np. hasła) przechowuj w **zmiennych środowiskowych** – to oddziela konfigurację od kodu. Pamiętaj: zmienne środowiskowe nie są mechanizmem bezpieczeństwa; w produkcji uzupełnij je o dedykowane zarządzanie sekretami (np. HashiCorp Vault, AWS Secrets Manager).

2.2.3 Lokalizacja i struktura katalogów PostgreSQL

Pliki konfiguracyjne PostgreSQL znajdują się w katalogu danych (PGDATA).

Główne pliki konfiguracyjne:

- `postgresql.conf` – podstawowa konfiguracja serwera (port, pamięć, logi)
- `pg_hba.conf` – kontrola dostępu (kto i skąd może się łączyć)
- `pg_ident.conf` – mapowanie użytkowników systemowych na role PostgreSQL

Domyślne lokalizacje:

Table 1: Domyślne lokalizacje

System	Katalog danych
Linux (RPM/DEB)	/var/lib/pgsql/data/ lub /var/lib/postgresql/*/main/
Linux (kompilacja źródłowa)	/usr/local/pgsql/data/
Windows	C:\Program Files\PostgreSQL\<version>\data\
Docker	/var/lib/postgresql/data/ (w kontenerze)

Zmiana katalogu danych:

```
# Inicjalizacja nowego katalogu
initdb -D /ścieżka/do/katalogu

# Uruchomienie z niestandardowym PGDATA
pg_ctl -D /ścieżka/do/katalogu start
```

Struktura katalogu danych:

```
$PGDATA/
├── postgresql.conf  # główny plik konfiguracyjny
├── pg_hba.conf      # polityka dostępu
├── pg_ident.conf    # mapowanie tożsamości
├── base/            # rzeczywiste bazy danych
├── global/          # tabele systemowe
├── log/             # logi serwera
└── pg_wal/          # Write-Ahead Log (WAL)
```

Sprawdzenie aktualnej lokalizacji:

```
SHOW data_directory;
SHOW config_file;
SHOW hba_file;
```

2.2.4 Środowiska i profile

Przełączanie środowisk PostgreSQL (dev/test/prod) realizuj przez:

- zmienne środowiskowe (np. PGHOST, PGPORT, PGDATABASE, PGUSER, PGPASSWORD),
- profile konfiguracyjne,
- osobne pliki .env dla każdego środowiska.

2.2.5 Automatyzacja

Zalecenia dla PostgreSQL:

- walidacja konfiguracji przy starcie za pomocą pg_isready,
- skrypty sprawdzające kompletność zmiennych środowiskowych,
- integracja z CI/CD (np. GitHub Actions, GitLab CI).

Dokumentację generuj automatycznie (Sphinx).

2.2.6 Przykłady**PostgreSQL – zmienne środowiskowe (.env)**

```
PGHOST=localhost
PGPORT=5432
PGDATABASE=aplikacja
```

(continues on next page)

(continued from previous page)

```
PGUSER=app_user
PGPASSWORD=${DB_PASSWORD}
```

PostgreSQL – URL połączenia

```
DATABASE_URL=postgresql://${PGUSER}:${PGPASSWORD}@${PGHOST}:${PGPORT}/${PGDATABASE}
```

PostgreSQL – sprawdzenie połączenia

```
pg_isready -h ${PGHOST} -p ${PGPORT} -U ${PGUSER} -d ${PGDATABASE}
```

2.2.7 Najlepsze praktyki

1. Nie przechowuj sekretów w repozytorium.
2. Stosuj `.env.example` zamiast rzeczywistych danych.
3. Używaj standardowych zmiennych środowiskowych PostgreSQL (PG*).
4. Waliduj konfigurację przed uruchomieniem (`pg_isready`).
5. W środowiskach produkcyjnych używaj połączeń SSL/TLS (`sslmode=require`).

2.2.8 Podsumowanie

Poprawna konfiguracja połączenia z PostgreSQL zwiększa bezpieczeństwo i przenośność aplikacji między środowiskami. Standardowe zmienne środowiskowe oraz narzędzia takie jak `pg_isready` ułatwiają automatyzację i walidację.

2.2.9 Szczegółowe omówienie plików konfiguracyjnych

Po omówieniu podstawowych wymagań środowiskowych można przejść do szczegółowej konfiguracji PostgreSQL. W kolejnych sekcjach przedstawiono najważniejsze parametry konfiguracyjne dostępne w plikach:

- `postgresql.conf` – konfiguracja działania serwera,
- `pg_hba.conf` – kontrola dostępu do bazy danych,
- `pg_ident.conf` – mapowanie użytkowników systemowych na role PostgreSQL.

2.2.10 postgresql.conf

Plik `postgresql.conf` jest głównym plikiem konfiguracyjnym klastra PostgreSQL. Zawiera ustawienia wpływające na działanie serwera, wydajność, bezpieczeństwo, logowanie oraz replikację.

Lokalizację aktywnego pliku można sprawdzić poleceniem:

```
SHOW config_file;
```

Najważniejsze parametry**Połączenia sieciowe****listen_addresses**

Określa adresy IP, na których PostgreSQL nasłuchuje połączeń.

Najczęstsze wartości:

- `localhost` – tylko połączenia lokalne,
- `*` – wszystkie interfejsy sieciowe,
- lista adresów, np. `'192.168.1.10,localhost'`.

port

Port TCP używany przez PostgreSQL.

Domyślna wartość:

```
port = 5432
```

max_connections

Maksymalna liczba jednoczesnych połączeń z bazą danych.
Zbyt wysoka wartość może zwiększać zużycie pamięci.

Pamięć i wydajność**shared_buffers**

Ilość pamięci RAM używanej przez PostgreSQL do buforowania danych.

Typowe wartości:

- 128MB – środowiska testowe,
- 25% pamięci RAM – środowiska produkcyjne.

work_mem

Ilość pamięci wykorzystywanej przez operacje sortowania i zapytania.

Parametr jest przydzielany dla pojedynczej operacji.

maintenance_work_mem

Pamięć używana podczas operacji administracyjnych, np.:

- VACUUM,
- CREATE INDEX,
- ALTER TABLE.

effective_cache_size

Szacowany rozmiar pamięci podręcznej dostępnej dla PostgreSQL.

Parametr pomaga optymalizatorowi zapytań wybierać odpowiednie plany wykonania.

Logowanie**logging_collector**

Włącza zapisywanie logów do plików.

Dostępne wartości:

- on,
- off.

log_directory

Katalog przechowywania logów.

log_filename

Wzorzec nazw plików logów.

Przykład:

```
log_filename = 'postgresql-%Y-%m-%d.log'
```

log_min_duration_statement

Określa minimalny czas wykonania zapytania (w ms), po którym zostanie ono zapisane w logach.

Przykład:

```
log_min_duration_statement = 1000
```

Wartość 1000 oznacza logowanie zapytań trwających dłużej niż 1 sekundę.

Bezpieczeństwo**password_encryption**

Algorytm szyfrowania haseł użytkowników.

Zalecana wartość:

```
password_encryption = scram-sha-256
```

ssl

Włącza obsługę połączeń SSL/TLS.

Dostępne wartości:

- on,
- off.

ssl_cert_file / ssl_key_file

Ścieżki do certyfikatu i klucza prywatnego SSL.

Replikacja i WAL**wal_level**

Określa poziom szczegółowości Write-Ahead Log (WAL).

Dostępne wartości:

- minimal,
- replica,
- logical.

max_wal_size

Maksymalny rozmiar plików WAL przed wykonaniem checkpointu.

archive_mode

Włącza archiwizację WAL.

archive_command

Polecenie wykonywane podczas archiwizacji plików WAL.

Przykładowa konfiguracja

```
listen_addresses = '*'
port = 5432
max_connections = 200

shared_buffers = 1GB
work_mem = 16MB
maintenance_work_mem = 256MB

logging_collector = on
log_directory = 'log'

ssl = on
password_encryption = scram-sha-256

wal_level = replica
max_wal_size = 2GB
```

Weryfikacja konfiguracji

Po zmianie parametrów konfigurację można przeładować bez restartu serwera:

```
SELECT pg_reload_conf();
```

Niektóre parametry wymagają pełnego restartu PostgreSQL.

Aktualne wartości parametrów można sprawdzić poleceniem:

```
SHOW shared_buffers;
SHOW wal_level;
SHOW max_connections;
```

2.2.11 pg_hba.conf

Plik `pg_hba.conf` (Host-Based Authentication) odpowiada za kontrolę dostępu do serwera PostgreSQL.

Definiuje:

- kto może połączyć się z bazą danych,
- z jakiego adresu IP,
- do jakiej bazy danych,
- przy użyciu jakiej metody uwierzytelniania.

Lokalizację aktywnego pliku można sprawdzić poleceniem:

```
SHOW hba_file;
```

Składnia wpisu

Każdy wpis w `pg_hba.conf` ma następującą strukturę:

TYPE	DATABASE	USER	ADDRESS	METHOD
------	----------	------	---------	--------

Znaczenie pól:

TYPE

Typ połączenia.

Dostępne wartości:

- `local` – połączenia lokalne (Unix socket),
- `host` – połączenia TCP/IP,
- `hostssl` – połączenia TCP/IP z SSL,
- `hostnossl` – połączenia TCP/IP bez SSL.

DATABASE

Nazwa bazy danych, do której użytkownik może się połączyć.

Możliwe wartości:

- konkretna nazwa bazy,
- `all` – wszystkie bazy,
- `sameuser` – baza o nazwie zgodnej z użytkownikiem.

USER

Użytkownik lub rola PostgreSQL.

ADDRESS

Adres IP lub zakres sieci w notacji CIDR.

Przykłady:

- `127.0.0.1/32`,
- `192.168.1.0/24`,
- `0.0.0.0/0` – wszystkie adresy.

METHOD

Metoda uwierzytelniania użytkownika.

Najczęściej używane metody uwierzytelniania

trust

Zezwala na połączenie bez uwierzytelniania.

Zalecane wyłącznie w środowiskach testowych.

reject

Odrzuca połączenie.

md5

Uwierzytelnianie przy użyciu hasła MD5.

Metoda starsza i mniej bezpieczna niż SCRAM.

scram-sha-256

Nowoczesna metoda uwierzytelniania oparta na SCRAM.

Zalecana dla nowych instalacji PostgreSQL.

peer

Uwierzytelnianie na podstawie użytkownika systemowego.

Najczęściej używane dla lokalnych połączeń Linux/Unix.

ident

Mapowanie użytkownika systemowego przy użyciu `pg_ident.conf`.

Przykładowa konfiguracja

```
# Połączenia lokalne
local  all                postgres                    peer

# Połączenia localhost IPv4
host   all                all                127.0.0.1/32          scram-sha-256

# Połączenia localhost IPv6
host   all                all                ::1/128              scram-sha-256

# Sieć lokalna
host   aplikacja          app_user          192.168.1.0/24        scram-sha-256
```

Najlepsze praktyki

1. Używaj `scram-sha-256` zamiast `md5`.
2. Ograniczaj dostęp do konkretnych adresów IP.
3. Nie używaj `trust` w środowiskach produkcyjnych.
4. Stosuj `hostssl` dla połączeń zdalnych.
5. Po zmianie konfiguracji wykonuj reload konfiguracji:

```
SELECT pg_reload_conf();
```

2.2.12 pg_ident.conf

Plik `pg_ident.conf` umożliwia mapowanie użytkowników systemowych na role PostgreSQL.

Najczęściej używany jest razem z metodami uwierzytelniania:

- `peer`,
- `ident`.

Lokalizację aktywnego pliku można sprawdzić poleceniem:

```
SHOW ident_file;
```

Zastosowanie

Mechanizm mapowania pozwala określić, który użytkownik systemowy może zostać powiązany z konkretną rolą PostgreSQL.

Jest to szczególnie przydatne:

- w środowiskach Linux/Unix,
- przy automatyzacji administracyjnej,
- dla lokalnych połączeń bez użycia haseł.

Składnia wpisu

MAPNAME	SYSTEM-USERNAME	PG-USERNAME
---------	-----------------	-------------

Znaczenie pól:

MAPNAME

Nazwa mapowania wykorzystywana w `pg_hba.conf`.

SYSTEM-USERNAME

Użytkownik systemu operacyjnego.

PG-USERNAME

Rola PostgreSQL, na którą użytkownik ma zostać zmapowany.

Przykładowa konfiguracja

# MAPNAME	SYSTEM-USERNAME	PG-USERNAME
admin_map	postgres	postgres
admin_map	deploy	db_admin
app_map	appuser	aplikacja

Przykład użycia w `pg_hba.conf`:

```
local    all        all        peer map=admin_map
```

W powyższym przykładzie użytkownicy systemowi zdefiniowani w `admin_map` mogą logować się jako odpowiadające im role PostgreSQL.

Najlepsze praktyki

1. Ograniczaj mapowania wyłącznie do wymaganych użytkowników.
2. Nie mapuj użytkowników systemowych na role superuser bez uzasadnienia.
3. Stosuj osobne role administracyjne i aplikacyjne.
4. Dokumentuj wszystkie mapowania użytkowników.
5. Regularnie przeglądaj konfigurację dostępu.

Opracowanie

Autor: Michał Kraus **Przedmiot:** Bazy danych **Data:** maj 2026

2.3 Kontrola i konserwacja

Autorzy

1. Paweł Łoćwin
2. Paweł Łosowski

2.3.1 Wprowadzenie i planowanie konserwacji

Współczesne systemy relacyjnych baz danych to wysoce skomplikowane środowiska, które wymagają ciągłego i proaktywnego nadzoru. Aby zagwarantować wysoką dostępność i niezawodność, niezbędne jest rygorystyczne planowanie procesów konserwacyjnych, które opiera się na odpowiedzi na trzy kluczowe pytania:

- **Kiedy?** Operacje inwazyjne (np. pełna przebudowa tabel i indeksów) powinny być planowane w tzw. oknach serwisowych, czyli w godzinach najniższego obciążenia systemu (zwykle w nocy lub w weekendy). Lżejsze prace konserwacyjne mogą odbywać się w tle.
- **W jakim zakresie?** Należy precyzyjnie określić, czy konserwacji wymaga cała instancja, pojedyncza baza danych, czy tylko wytypowane, najszybciej rosnące tabele. Skupienie się na najbardziej obciążonych obszarach oszczędza zasoby sprzętowe.

- **Jak?** Konserwacja powinna być maksymalnie zautomatyzowana (np. z wykorzystaniem systemowego demona CRON lub rozszerzenia `pg_cron` w PostgreSQL). Ręczne interwencje administratora powinny być zarezerwowane wyłącznie dla sytuacji awaryjnych.

Uwaga dla środowisk SQLite: W przeciwieństwie do potężnego PostgreSQL, SQLite jako baza plikowa i wbudowana (embedded) nie posiada demonów działających w tle, więc pojęcie “planowania konserwacji” sprowadza się najczęściej do ręcznego wywoływania skryptów optymalizujących (np. z poziomu aplikacji klienckiej) w momentach, gdy plik bazy nie jest zablokowany.

2.3.2 Zarządzanie stanem serwera i sesjami

Podstawą kontroli nad środowiskiem bazodanowym w architekturze klient-serwer (takiej jak PostgreSQL) jest zarządzanie cyklem życia samej usługi. Uruchamianie, zatrzymywanie i restartowanie serwera bazy danych realizuje się najczęściej za pośrednictwem menedżera usług systemu operacyjnego (np. polecenia `systemctl start/stop/restart postgresql`) lub za pomocą dedykowanego narzędzia wiersza poleceń `pg_ctl`.

Podczas wdrażania krytycznych aktualizacji lub w sytuacjach awaryjnych, konieczne jest sprawne zarządzanie ruchem użytkowników:

- **Zapobieganie nowym połączeniom:** Administrator może tymczasowo zablokować możliwość logowania się do konkretnej bazy danych poprzez zmianę jej metadanych. Pozwala to na “odcięcie” aplikacji bez wyłączania całego serwera.
- **Ograniczanie i rozłączanie użytkowników:** Istniejące, aktywne sesje, które np. zawiesiły się lub blokują ważne operacje, można siłowo przerwać, zwalniając tym samym zablokowane zasoby.

```
-- Zapobieganie nawiązywaniu nowych połączeń do bazy danych (PostgreSQL)
ALTER DATABASE biblioteka_db ALLOW_CONNECTIONS = false;

-- Siłowe rozłączenie wszystkich aktywnych użytkowników (poza aktualną sesją)
SELECT pg_terminate_backend(pid)
FROM pg_stat_activity
WHERE datname = 'biblioteka_db' AND pid <> pg_backend_pid();
```

Uwaga dla środowisk SQLite: Ponieważ SQLite nie posiada procesu serwera zarządzającego użytkownikami sieciowymi, nie ma tu koncepcji “zabijania sesji” czy blokowania nowych połączeń sieciowych. Dostęp jest kontrolowany wyłącznie na poziomie blokad plików systemu operacyjnego (file locks).

2.3.3 Proces Vacuum

Architektura PostgreSQL opiera się na mechanizmie MVCC (Multi-Version Concurrency Control). Oznacza to, że instrukcje aktualizacji (UPDATE) i usuwania (DELETE) nie niszczą fizycznie starych danych, lecz tworzą tzw. “martwe krotki” (dead tuples). Do zarządzania nimi służy proces czyszczenia:

- **VACUUM:** Standardowa operacja zwalniająca miejsce po martwych wierszach, czyniąca je dostępnymi dla nowych danych. Nie zmniejsza ona fizycznego rozmiaru pliku na dysku i nie blokuje możliwości jednoczesnego odczytu i zapisu tabeli.
- **VACUUM FULL:** Inwazyjna i ciężka operacja, która fizycznie przebudowuje tabelę, faktycznie zmniejszając jej rozmiar i zwracając wolne miejsce do systemu operacyjnego. Wymaga ekskluzywnej blokady (lock), całkowicie odcinając aplikację od modyfikowanej tabeli na czas trwania procesu.
- **AUTOVACUUM:** Zautomatyzowany proces działający w tle, który regularnie monitoruje poziom zmian w tabelach i samoczynnie wyzwała standardowy Vacuum, zapobiegając niekontrolowanemu puchnięciu bazy danych (table bloat).

Uwaga dla środowisk SQLite: SQLite w odpowiedzi na operację DELETE po prostu oznacza miejsce jako “wolne” w pliku. Aby fizycznie skurczyć plik, należy wywołać unikalne polecenie VACUUM (odpowiednik VACUUM FULL z PostgreSQL). Możliwe jest również włączenie trybu PRAGMA auto_vacuum = FULL, jednak ze względów wydajnościowych narzuca on znaczny koszt przy każdej modyfikacji.

2.3.4 Zarządzanie transakcjami i schematy (SCHEMA)

Kolejnym aspektem administracji jest dbanie o logiczną strukturę bazy oraz spójność operacji.

SCHEMA (Schematy) to wirtualne przestrzenie nazw (przypominające katalogi w systemie operacyjnym), służące do logicznego grupowania tabel, widoków i funkcji. Warto nadmienić, że mechanizm logicznych schematów znany z PostgreSQL nie występuje natywnie w SQLite (istnieje tam komenda ATTACH do dołączania innych plików baz danych jako schematów, ale działa to na innej zasadzie). Główne zastosowania schematów w PostgreSQL to:

- Izolacja danych różnych mikrousług lub aplikacji wewnątrz jednej wspólnej bazy danych.
- Implementacja architektury wielodostępowej (multi-tenant), gdzie każdy klient aplikacji obsługiwany jest w dedykowanym, oddzielnym schemacie.
- Ułatwienie i zbiorcze zarządzanie uprawnieniami dostępu.

Zarządzanie transakcjami to proces kontrolowania instrukcji (BEGIN, COMMIT, ROLLBACK) w taki sposób, aby zachować zgodność z regułami ACID. Administrator musi zwracać szczególną uwagę na zapobieganie tzw. długim transakcjom (long-running transactions). Otwarta i porzucona transakcja powstrzymuje proces Vacuum przed usuwaniem martwych krotek, co prowadzi do drastycznego spadku wydajności całej bazy (dotyczy to tak samo PostgreSQL, jak i SQLite, gdzie długa transakcja zapisu blokuje cały plik bazy).

```
-- Przykład logiki transakcyjnej z wykorzystaniem konkretnego schematu
BEGIN;
INSERT INTO finanse.historia (kwota, opis) VALUES (200, 'Opłata serwisowa');
UPDATE finanse.konta SET saldo = saldo - 200 WHERE id_klienta = 5;
COMMIT;
```

2.3.5 Zarządzanie indeksami

Indeksy są fundamentalne dla wydajnego wyszukiwania danych, jednak ich utrzymanie kosztuje — spowalniają one modyfikacje danych (DML) i konsumują miejsce na dysku.

Prawidłowe zarządzanie indeksami polega na stałym monitorowaniu ich wykorzystania. Należy identyfikować i usuwać indeksy, które nie są wykorzystywane przez planistę zapytań (tzw. unused indexes). Dodatkowo, podobnie jak tabele, indeksy ulegają fragmentacji. Z tego powodu administrator powinien regularnie planować operację **REINDEX**, która od nowa buduje strukturę drzewa indeksu, przywracając mu optymalny rozmiar i maksymalną wydajność. W systemach PostgreSQL o wysokiej dostępności stosuje się przebudowę w trybie CONCURRENTLY, która nie przerywa pracy aplikacji (SQLite posiada jedynie standardowe polecenie REINDEX, blokujące tabele).

2.3.6 Podsumowanie

Prawidłowa kontrola i konserwacja środowiska bazodanowego to wielowymiarowy proces, wymagający głębokiego zrozumienia zarówno architektury logicznej, jak i fizycznej silnika.

Stabilność i wydajność dużych systemów relacyjnych (PostgreSQL) zależy w równej mierze od prawidłowego zarządzania stanem usługi i połączeniami sieciowymi, dogłębnego planowania okien serwisowych, jak i proaktywnego zarządzania mechanizmem AutoVacuum. Z kolei w środowiskach wbudowanych (SQLite) ciężar ten przenosi się na precyzyjne zarządzanie blokadami plików operacyjnych oraz manualną defragmentację przestrzeni. Niezależnie od wybranego silnika, regularne dbanie o kondycję indeksów i izolacja transakcyjna pozwalają na zbudowanie stabilnego systemu odpornego na obciążenia w trudnych, produkcyjnych warunkach.

2.4 Monitorowanie i diagnostyka

Autorzy

1. Oskar Wrona
2. Kamil Lewandowski
3. Adam Tarkowski

2.4.1 Wstęp

Monitorowanie i diagnostyka baz danych obejmują zestaw działań, które pozwalają administratorowi lub zespołowi utrzymania sprawdzić, czy system działa poprawnie, bezpiecznie i wydajnie. W praktyce nie chodzi jedynie o obserwowanie pojedynczych parametrów serwera, ale o łączenie informacji pochodzących z wielu warstw: samej bazy danych, systemu operacyjnego, aplikacji oraz narzędzi zewnętrznych. Dzięki temu można szybciej wykrywać przeciążenia, błędy konfiguracji, problemy z zapytaniami SQL, nieprawidłowe uprawnienia oraz awarie sprzętowe lub sieciowe.

W systemach bazodanowych szczególnie ważne jest to, że wiele problemów nie jest od razu widocznych dla użytkownika końcowego. Przykładowo zapytania mogą stopniowo wykonywać się coraz wolniej, liczba aktywnych sesji może rosnąć, a logi mogą zawierać powtarzające się ostrzeżenia. Bez monitorowania takie sygnały są łatwe do przeoczenia. Diagnostyka pozwala natomiast przejść od samego wykrycia problemu do ustalenia jego przyczyny, np. przez analizę sesji, blokad, planów zapytań, dzienników błędów lub zużycia zasobów systemowych.

2.4.2 Sesje i użytkownicy

Jednym z podstawowych obszarów monitorowania bazy danych jest obserwowanie aktywnych sesji i użytkowników. Sesja oznacza połączenie użytkownika lub aplikacji z serwerem bazy danych. W ramach sesji wykonywane są zapytania, transakcje oraz operacje administracyjne. Analiza sesji pozwala sprawdzić między innymi, kto jest aktualnie połączony z bazą, z jakiego hosta pochodzi połączenie, jakie zapytanie jest wykonywane oraz czy sesja jest aktywna, bezczynna albo zablokowana.

W wielu systemach zarządzania bazami danych dostępne są specjalne widoki lub narzędzia administracyjne służące do obserwowania bieżącej aktywności. Na przykład PostgreSQL udostępnia mechanizmy monitorowania aktywności bazy danych oraz widoki statystyczne, takie jak `pg_stat_activity`, które pozwalają sprawdzać informacje o procesach serwera i wykonywanych zapytaniach¹. W MySQL podobną rolę może pełnić Performance Schema, które gromadzi dane diagnostyczne o pracy serwera i może być wykorzystywane do analizy wydajności². Monitorowanie sesji jest istotne zarówno z punktu widzenia wydajności, jak i bezpieczeństwa. Zbyt duża liczba jednoczesnych połączeń może prowadzić do przeciążenia serwera, a długo działające transakcje mogą blokować inne operacje. Z kolei nietypowe połączenia, np. z nieznanych adresów lub wykonywane poza normalnymi godzinami pracy, mogą wskazywać na błąd konfiguracji albo próbę nieautoryzowanego dostępu.

2.4.3 Śledzenie dostępu użytkowników do tabel

Kolejnym ważnym elementem jest kontrola tego, którzy użytkownicy uzyskują dostęp do określonych tabel i jakie operacje wykonują. W bazach danych przechowywane są często dane wrażliwe, dlatego sama informacja o tym, że użytkownik posiada konto w systemie, nie jest wystarczająca. Należy również wiedzieć, czy jego działania są zgodne z nadanymi uprawnieniami oraz z zasadami bezpieczeństwa obowiązującymi w organizacji.

Śledzenie dostępu może obejmować operacje odczytu danych, modyfikacji rekordów, usuwania danych, tworzenia nowych obiektów oraz zmian w uprawnieniach. W praktyce realizuje się to za pomocą mechanizmów audytu, dzienników zapytań, wyzwalaczy, narzędzi klasy Database Activity Monitoring lub funkcji wbudowanych w konkretny system bazodanowy. Przykładowo Oracle Database udostępnia mechanizmy audytu, a nowsze wersje zalecają korzystanie z unified auditing zamiast starszego audytu tradycyjnego³.

Takie śledzenie jest potrzebne nie tylko przy wykrywaniu incydentów bezpieczeństwa. Pomaga również w spełnianiu wymagań formalnych, analizie błędów użytkowników oraz odtwarzaniu przebiegu zdarzeń po awarii. Przykładowo, jeśli w tabeli pojawiły się nieprawidłowe dane, historia dostępu może pomóc ustalić, kiedy doszło do zmiany i która aplikacja lub który użytkownik ją wykonał.

¹ PostgreSQL Documentation, *Monitoring Database Activity* oraz *The Cumulative Statistics System*, <https://www.postgresql.org/docs/current/monitoring.html>

² MySQL Documentation, *Using the Performance Schema to Diagnose Problems*, <https://dev.mysql.com/doc/refman/8.2/en/performance-schema-examples.html>

³ Oracle Documentation, *AUDIT_TRAIL* oraz informacje o unified auditing, https://docs.oracle.com/en/database/oracle/oracle-database/21/refrn/AUDIT_TRAIL.html

2.4.4 Błędy dziennika, logi i raporty

Dzienniki zdarzeń, czyli logi, są jednym z najważniejszych źródeł informacji diagnostycznych. Serwer bazy danych może zapisywać w nich komunikaty o błędach, ostrzeżeniach, nieudanych logowaniach, problemach z zapytaniem, restartach, zmianach konfiguracji lub przekroczeniu określonych progów wydajnościowych. Analiza logów pozwala wykrywać problemy, które nie zawsze są widoczne w aplikacji klienckiej.

W zależności od systemu bazodanowego logi mogą mieć różną postać. PostgreSQL obsługuje różne metody logowania komunikatów serwera, między innymi `stderr`, `csvlog`, `jsonlog` oraz `syslog`; w systemie Windows możliwe jest także użycie dziennika zdarzeń⁴. MySQL udostępnia między innymi `general query log`, który może rejestrować połączenia i zapytania otrzymywane przez serwer, oraz `slow query log`, używany do identyfikowania zapytań wykonujących się zbyt długo⁵.

Raporty diagnostyczne są rozwinięciem surowych logów. Zamiast ręcznie analizować tysiące wpisów, administrator może korzystać z raportów podsumowujących liczbę błędów, najwolniejsze zapytania, częstotliwość nieudanych logowań lub zmiany obciążenia w czasie. Raporty są szczególnie przydatne w pracy zespołowej, ponieważ pozwalają dokumentować problemy i porównywać stan systemu przed oraz po wprowadzeniu zmian optymalizacyjnych.

2.4.5 Monitorowanie na poziomie systemu operacyjnego

Baza danych działa na konkretnym systemie operacyjnym i korzysta z jego zasobów. Dlatego diagnostyka nie może ograniczać się wyłącznie do samego silnika bazy danych. Wydajność zapytań może zależeć od procesora, pamięci RAM, operacji wejścia/wyjścia na dysku, sieci, liczby procesów oraz ogólnego obciążenia systemu.

W systemach Linux często wykorzystuje się narzędzia takie jak `iostat`, `htop` oraz `vmstat`. Narzędzie `iostat` pozwala obserwować wykorzystanie procesora i urządzeń wejścia/wyjścia, co jest szczególnie ważne przy bazach danych intensywnie korzystających z dysku. `htop` umożliwia wygodne śledzenie procesów i zużycia zasobów w czasie rzeczywistym. `vmstat` pokazuje między innymi informacje o pamięci, procesach, wymianie danych z dyskiem oraz obciążeniu procesora.

W systemach Microsoft Windows podobną rolę pełnią Menedżer zadań oraz Monitor wydajności. Pozwalają one obserwować zużycie procesora, pamięci, dysków, sieci oraz usług działających w tle. W środowiskach produkcyjnych takie dane są często zbierane automatycznie i zestawiane z metrykami pochodzącymi z bazy danych. Dzięki temu można odróżnić problem wynikający z nieoptymalnego zapytania SQL od problemu spowodowanego np. przeciążeniem dysku lub brakiem pamięci.

2.4.6 Monitorowanie serwera

Ostatnim poziomem jest monitorowanie całego serwera lub infrastruktury, w której działa baza danych. Obejmuje ono dostępność usług, czas odpowiedzi, zużycie zasobów, stan dysków, wykorzystanie sieci, działanie kopii zapasowych oraz wysyłanie powiadomień w przypadku awarii. W tym celu stosuje się narzędzia takie jak Nagios, Grafana, Prometheus, Zabbix lub inne systemy monitoringu.

Nagios jest przykładem narzędzia używanego do monitorowania usług i hostów, sprawdzania ich stanu oraz informowania administratorów o problemach⁶. Grafana z kolei służy przede wszystkim do wizualizacji danych monitorujących w formie dashboardów. Panele Grafany mogą prezentować metryki w postaci wykresów, tabel i innych wizualizacji, a moduł alertów pozwala reagować na określone zdarzenia lub przekroczenie progów⁷.

Monitorowanie serwera jest szczególnie ważne w środowiskach, w których baza danych stanowi element większego systemu. Awaria bazy może wynikać nie tylko z błędu samego silnika, ale również z problemu sieciowego, zapełnionego dysku, braku pamięci, błędnej konfiguracji systemu lub nieudanego wdrożenia aplikacji. Centralne monitorowanie pozwala zebrać te informacje w jednym miejscu i szybciej wskazać źródło problemu.

⁴ PostgreSQL Documentation, *Error Reporting and Logging*, <https://www.postgresql.org/docs/current/runtime-config-logging.html>

⁵ MySQL Documentation, *The General Query Log* oraz *The Slow Query Log*, <https://dev.mysql.com/doc/en/query-log.html>

⁶ Nagios Open Source Documentation, <https://www.nagios.org/documentation/>

⁷ Grafana Documentation, *Dashboards overview* oraz *Grafana Alerting*, <https://grafana.com/docs/grafana/latest/fundamentals/dashboards-overview/>

2.4.7 Znaczenie monitorowania i diagnostyki w bazach danych

Monitorowanie i diagnostyka są ważne, ponieważ wpływają na trzy kluczowe obszary: dostępność, wydajność i bezpieczeństwo danych. Dostępność oznacza, że baza danych jest osiągalna dla użytkowników i aplikacji. Wydajność oznacza, że zapytania wykonują się w akceptowalnym czasie. Bezpieczeństwo oznacza natomiast, że dostęp do danych jest kontrolowany, a podejrzone działania mogą zostać wykryte i przeanalizowane.

W dobrze utrzymanym środowisku bazodanowym monitorowanie powinno działać stale, a nie dopiero wtedy, gdy pojawi się awaria. Regularne obserwowanie sesji, logów, dostępu do tabel, parametrów systemu operacyjnego i stanu serwera pozwala wykrywać problemy na wczesnym etapie. Diagnostyka pozwala natomiast ustalić przyczyny tych problemów i zaplanować odpowiednie działania, takie jak optymalizacja zapytań, zmiana konfiguracji, zwiększenie zasobów, poprawa uprawnień lub wdrożenie dodatkowych zabezpieczeń.

2.4.8 Podsumowanie

Monitorowanie i diagnostyka baz danych tworzą wielowarstwowy proces kontroli działania systemu. Na poziomie bazy danych analizuje się sesje, użytkowników, zapytania, blokady, dostęp do tabel oraz logi. Na poziomie systemu operacyjnego obserwuje się zużycie procesora, pamięci, dysku i sieci. Na poziomie serwera wykorzystuje się narzędzia monitorujące, które zbierają metryki, tworzą wykresy oraz wysyłają alerty. Dopiero połączenie tych perspektyw daje pełny obraz stanu systemu i pozwala skutecznie reagować na błędy, spadki wydajności oraz potencjalne incydenty bezpieczeństwa.

2.4.9 Źródła

2.5 Wydajność, skalowanie i replikacja

Autorzy

1. Olaf Chomicki
2. Konrad Machowski
3. Wiktor Wydrzyński

2.5.1 Wydajność, skalowanie i replikacja

Współczesne systemy bazodanowe muszą sprostać wymaganiom związanym z wykładniczym przyrostem wolumenu danych oraz stale rosnącą liczbą jednoczesnych użytkowników. W tym kontekście kluczowe stają się zrozumienie trzech fundamentalnych pojęć: wydajności, skalowania i replikacji. Wydajność odnosi się do szybkości przetwarzania pojedynczych transakcji i zapytań przy optymalnym wykorzystaniu dostępnych zasobów.

Gdy optymalizacja kodu przestaje wystarczać, konieczne stają się skalowanie – pionowe (zwiększanie mocy pojedynczego serwera) lub poziome (rozpraszanie obciążenia na wiele maszyn). Replikacja z kolei stanowi most łączący skalowanie odczytów z zapewnieniem wysokiej dostępności i odporności systemu na awarie sprzętowe.

2.5.2 Kontrola i buforowanie połączeń z bazą danych

Nawiązywanie nowego połączenia z bazą danych jest operacją niezwykle kosztowną procesorowo i czasowo, ponieważ wymaga uwierzytelnienia użytkownika, alokacji pamięci oraz ustanowienia sesji sieciowej. W systemach o dużym natężeniu ruchu brak kontroli nad liczbą połączeń prowadzi do szybkiego wyczerpania zasobów serwera. Rozwiązaniem tego problemu jest buforowanie połączeń (Connection Pooling). Mechanizm ten polega na utrzymywaniu stałej puli otwartych, gotowych do użycia połączeń, które są wielokrotnie współdzielone przez aplikację. Narzędzia takie jak PgBouncer (dla PostgreSQL) czy wbudowane pule w serwerach aplikacji drastycznie zmniejszają narzut overhead, stabilizując czas reakcji bazy danych pod dużym obciążeniem.

Przykład konfiguracji PgBouncer (fragment pliku `pgbouncer.ini`):

```
[databases]
moja_baza = host=127.0.0.1 port=5432 dbname=produkcja

[pgbouncer]
```

(continues on next page)

(continued from previous page)

```
listen_port = 6432
listen_addr = *
auth_type = md5
auth_file = /etc/pgbouncer/userlist.txt
pool_mode = transaction
max_client_conn = 1000
default_pool_size = 20
```

2.5.3 INDEX i CLUSTER

Indeksowanie to podstawowa technika optymalizacji zapytań o charakterze odczytowym. Instrukcja INDEX tworzy pomocniczą strukturę danych (najczęściej w formie B-drzewa), która pozwala silnikowi bazy danych na błyskawiczne odnalezienie żądanych wierszy bez konieczności kosztownego przeszukiwania całej tabeli (Full Table Scan). Z kolei polecenie CLUSTER idzie o krok dalej – fizycznie reorganizuje strukturę danych na dysku w taki sposób, aby kolejność wierszy w tabeli dokładnie odpowiadała kolejności indeksu. Powoduje to, że powiązane rekordy są składowane obok siebie w tych samych blokach pamięci, co radykalnie przyspiesza zapytania zakresowe (Range Queries).

Przykład użycia poleceń w PostgreSQL:

```
-- 1. Tworzenie standardowego indeksu B-drzewa na kolumnie data_zamowienia
CREATE INDEX idx_zamowienia_data ON zamowienia (data_zamowienia);

-- 2. Fizyczne przemieszczenie danych na dysku według kolejności indeksu
CLUSTER zamowienia USING idx_zamowienia_data;
```

2.5.4 Rola i zastosowanie replikacji

Replikacja polega na permanentnym kopiowaniu i synchronizowaniu danych z głównego węzła bazy danych (Primary/Master) na węzły pomocnicze (Replica/Slave). Jej rola w architekturze systemów IT jest wieloaspektowa. Po pierwsze, gwarantuje wysoką dostępność (High Availability) – w przypadku fizycznej awarii Mastera, jedna z replik może automatycznie przejąć jego funkcje (proces Failover).

Po drugie, umożliwia skalowanie odczytów (Read Scalability). Przekierowanie operacji modyfikujących (INSERT, UPDATE) do węzła głównego, a operacji odczytu oraz generowania ciężkich raportów na repliki pozwala na znaczne obciążenie głównego serwera.

2.5.5 Oprogramowanie i zaimplementowane mechanizmy replikacji

Mechanizmy replikacji mogą być realizowane na poziomie silnika bazy danych lub za pomocą zewnętrznego oprogramowania. Wyróżnia się dwa główne podejścia pod kątem transmisji danych: replikację opartą na logu (WAL - Write-Ahead Logging) oraz replikację logiczną. Ze względu na synchronizację, proces ten może przebiegać synchronicznie lub asynchronicznie.

Przykład konfiguracji replikacji logicznej (wbudowanej w PostgreSQL):

```
-- KROK 1: Uruchomienie na węźle głównym (Primary) - zdefiniowanie publikacji
CREATE PUBLICATION pub_dane_sprzedazy FOR TABLE zamowienia, klienci;

-- KROK 2: Uruchomienie na węźle pomocniczym (Replica) - subskrypcja danych
CREATE SUBSCRIPTION sub_dane_sprzedazy
CONNECTION 'host=192.168.1.50 port=5432 dbname=produkcja user=repl_user_
↳password=secret'
PUBLICATION pub_dane_sprzedazy;
```

2.5.6 Limity systemu oraz ograniczanie dostępu użytkowników

Bezpieczeństwo i stabilność bazy danych wymagają rygorystycznego zarządzania limitami systemowymi oraz uprawnieniami. Zbyt duża swoboda przyznana użytkownikom lub procesom aplikacyjnym może doprowadzić do celowego bądź przypadkowego unieruchomienia systemu.

Przykład nakładania restrykcji i limitów zasobów w PostgreSQL:

```
-- 1. Ograniczenie maksymalnej liczby jednoczesnych połączeń dla danej roli
ALTER ROLE rola_analitik CONNECTION LIMIT 5;

-- 2. Ustawienie maksymalnego czasu wykonywania pojedynczego zapytania (np. 30 sekund)
ALTER ROLE rola_analitik SET statement_timeout = '30s';

-- 3. Nadanie uprawnień wyłącznie do odczytu danych w wybranym schemacie
REVOKE ALL ON ALL TABLES IN SCHEMA public FROM rola_analitik;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO rola_analitik;
```

2.5.7 Testy wydajności sprzętu na poziomie systemu operacyjnego

Przed wdrożeniem produkcyjnym bazy danych kluczowe jest przeprowadzenie niezależnych testów wydajnościowych podzespołów sprzętowych (Benchmarków) bezpośrednio na poziomie systemu operacyjnego, aby wykłuczyć błędy konfiguracyjne środowiska.

Przykłady poleceń diagnostycznych i testowych w systemie Linux:

```
# 1. Test wydajności procesora (CPU) za pomocą sysbench
sysbench cpu --cpu-max-prime=20000 run

# 2. Test przepustowości i alokacji pamięci RAM
sysbench memory --memory-block-size=1M --memory-total-size=10G run

# 3. Zaawansowany test wydajności dysku (IOPS i opóźnienia) narzędziem fio
# Symulacja losowego odczytu/zapisu (Random R/W) specyficznego dla baz danych
fio --name=test_bazy --ioengine=libaio --rw=randrw --bs=4k \
    --size=2G --numjobs=2 --runtime=30 --time_based \
    --filename=/var/lib/postgresql/test_io.file --group_reporting
```

2.6 Partycjonowanie danych

Autorzy

1. Michał Bystrzak
2. Damian Dominiak

2.6.1 Wstęp i istota problemu

W dobie systemów informatycznych przetwarzających gigabajty lub terabajty informacji (tzw. VLDB - Very Large Databases), tradycyjne podejście do składowania danych staje się wysoce niewydajne. Kiedy pojedyncza tabela rozrasta się do setek milionów rekordów, fizyczne ograniczenia sprzętowe oraz narzuty na zarządzanie indeksami powodują drastyczny spadek wydajności. Partycjonowanie to zaawansowana technika architektoniczna, polegająca na logicznym podziale jednej, ogromnej tabeli na mniejsze, odseparowane fizycznie fragmenty nazywane partycjami. Z perspektywy aplikacji docelowej oraz zapytań SQL, tabela wciąż prezentuje się i zachowuje jak spójna, pojedyncza relacja.

2.6.2 Cele stosowania partycjonowania danych

Implementacja mechanizmu partycjonowania w systemach relacyjnych jest podyktowana chęcią rozwiązania konkretnych problemów wydajnościowych i administracyjnych. Główne cele to:

- **Zwiększenie wydajności zapytań (Query Performance):** Fundamentalnym celem jest eliminacja konieczności skanowania całej tabeli (Full Table Scan). Dzięki mechanizmowi *Partition Pruning* (odcinanie partycji), planista zapytań analizuje warunki zdefiniowane w klauzuli WHERE i kieruje odczyt wyłącznie do tych fizycznych plików, które mogą zawierać poszukiwane dane.
- **Optymalizacja procesów utrzymaniowych (Maintenance):** Olbrzymie tabele są trudne w utrzymaniu. Operacje takie jak VACUUM, REINDEX czy odtwarzanie kopii zapasowych mogą blokować zasoby przez wiele godzin. Partycjonowanie pozwala na wykonywanie tych operacji selektywnie, tylko na tych partycjach, które uległy modyfikacji.
- **Zarządzanie cyklem życia danych (Data Lifecycle Management):** W przypadku danych historycznych (np. logów), najszybszą metodą ich usunięcia jest operacja DROP TABLE na starej partycji. Jest to operacja na metadanych, trwająca ułamki sekund, w przeciwieństwie do standardowego DELETE, które w architekturze MVCC (Multi-Version Concurrency Control) generuje ogromne ilości “martwych krotek” (dead tuples) i obciąża dziennik transakcji (WAL).

2.6.3 Zastosowanie partycjonowania

Technika ta nie jest uniwersalnym rozwiązaniem dla każdej bazy danych, lecz sprawdza się w specyficznych, wymagających scenariuszach:

- **Hurtownie danych (OLAP):** Systemy analityczne gromadzące dane historyczne. Partycjonowanie pozwala tam na błyskawiczne agregowanie danych (np. sumowanie przychodów z konkretnego miesiąca) bez wczytywania do pamięci RAM danych z całej dekady.
- **Systemy transakcyjne (OLTP) o wysokim przyroście:** Aplikacje rejestrujące logi systemowe, odczyty z czujników IoT, dane telemetryczne czy rejestry transakcji finansowych (Time-Series Data), gdzie każdego dnia przybywają miliony nowych wierszy.
- **Implementacja Storage Tiering:** Możliwość fizycznego rozdzielenia danych na różne nośniki. Najnowsze i najczęściej odpytywane partycje (Hot Data) umieszcza się na ultra-szybkich macierzach NVMe, podczas gdy starsze, rzadko używane partycje (Cold Data) przenosi się na tańsze wolumeny dyskowe (HDD), co znacząco optymalizuje koszty infrastruktury.

2.6.4 Zalety i wady partycjonowania

Jak każda technika optymalizacyjna, partycjonowanie stanowi kompromis inżynierski (Trade-off).

Zalety:

- Spektakularny wzrost wydajności operacji odczytu, o ile zapytanie wykorzystuje klucz partycjonowania.
- Optymalne wykorzystanie pamięci operacyjnej (RAM) – indeksy dla poszczególnych partycji są znacznie mniejsze i z większym prawdopodobieństwem zmieszczą się w całości w pamięci podręcznej (Cache).
- Możliwość bezinwazyjnego odłączania i dołączania gigabajtów danych niemal w czasie rzeczywistym.

Wady:

- Zwiększony narzut (overhead) na planistę zapytań. Przy bardzo dużej liczbie partycji (np. tysiące fragmentów), czas samego planowania zapytania przez silnik bazy może być dłuższy niż czas jego wykonania.
- Ograniczenia strukturalne: w wielu systemach DBMS (w tym w PostgreSQL) globalne klucze główne (Primary Keys) i ograniczenia unikalności (UNIQUE) muszą z założenia zawierać w sobie kolumnę będącą kluczem partycjonowania, co bywa trudne do pogodzenia z logiką biznesową aplikacji.
- Spadek wydajności w przypadku zapytań omijających klucz partycjonowania – silnik musi wówczas odpytać każdą fizyczną partycję z osobna.

2.6.5 Implementacja partycjonowania w PostgreSQL

Począwszy od wersji 10, PostgreSQL oferuje natywne wsparcie dla partycjonowania deklaratywnego. Eliminuje to konieczność ręcznego pisania skomplikowanych wyzwalaczy czy reguł (rules). Silnik obsługuje trzy główne strategie przydziału danych:

- **Zakresowe (RANGE):** Dane są dzielone na podstawie ciągłych przedziałów wartości. Najczęściej wykorzystywanym kluczem są typy daty i czasu (np. jedna partycja dla 2025-01-01 do 2025-01-31) lub sekwencje numeryczne.
- **Listowe (LIST):** Dane są dystrybuowane na podstawie konkretnych, dyskretnych wartości zdefiniowanych w tablicy. Metoda ta idealnie sprawdza się w przypadku podziału na kategorie biznesowe, takie jak "Status_Zlecenia" (osobna partycja dla wartości Zakończone, W trakcie) lub regiony geograficzne.
- **Haszujące (HASH):** Wiersze są rozdzielane na zadaną liczbę partycji (modułów) z wykorzystaniem zaawansowanej, matematycznej funkcji skrótu. Strategia ta nie służy przyspieszaniu zapytań zakresowych, lecz idealnie równomiernemu rozłożeniu ciężaru I/O podczas masowych operacji wstawiania (INSERT) w systemach wielodyskowych.

2.6.6 Ograniczenia i obejście w środowisku SQLite

W przeciwieństwie do potężnych, serwerowych silników RDBMS takich jak PostgreSQL, lekka i plikowa biblioteka SQLite **nie posiada** natywnego mechanizmu partycjonowania deklaratywnego. Wynika to z jej osadzonej (embedded) architektury.

Niemniej jednak, efekt partycjonowania w SQLite można emulować za pomocą programistycznego podejścia warstwowego: 1. **Podział fizyczny:** Utworzenie wielu mniejszych, niezależnych tabel (np. zlecenia_2025, zlecenia_2026). 2. **Warstwa logiczna:** Stworzenie wspólnego widoku (VIEW), który scala wszystkie te tabele z wykorzystaniem operatora UNION ALL. Umożliwia to wykonywanie zapytań SELECT na jednej strukturze. 3. **Zarządzanie modyfikacjami:** Obsługa instrukcji DML (INSERT, UPDATE, DELETE) wymaga zdefiniowania na widoku zestawu wyzwalaczy (TRIGGERS INSTEAD OF). Wyzwalacze te muszą przechwytywać dane w locie, analizować klucz (np. datę) i dynamicznie przekierowywać operację do odpowiedniej fizycznej pod-tabeli.

Choć to rozwiązanie jest funkcjonalne, należy pamiętać, że w SQLite każdorazowe uruchamianie wyzwalaczy generuje zauważalny narzut wydajnościowy w porównaniu do natywnego mechanizmu w PostgreSQL.

2.7 Bezpieczeństwo Baz Danych

Dział 7 przeglądu literaturowego poświęcony architekturze bezpieczeństwa, mechanizmom ochrony oraz analizie podatności w nowoczesnych systemach zarządzania bazami danych.

Autorzy

1. Oskar Mąka
2. Daniel Szokało
3. Dawid Szokało

2.7.1 Model bezpieczeństwa CIA i ochrona infrastruktury

Rozważając bezpieczeństwo systemów bazodanowych, należy spojrzeć na problem warstwowo. Najniższą, a zarazem najbardziej fundamentalną warstwą jest ochrona samej infrastruktury sprzętowej i sieciowej, na której operuje silnik bazy danych (DBMS). Zanim przejdziemy do szczegółowych konfiguracji silnika, konieczne jest zdefiniowanie nadrzędnych celów bezpieczeństwa oraz zapewnienie izolacji środowiska uruchomieniowego.

Triada CIA w środowisku bazodanowym

Model CIA (ang. *Confidentiality, Integrity, Availability*) to klasyczny paradygmat bezpieczeństwa informacji. W kontekście relacyjnych i nierelacyjnych baz danych każdy z tych elementów realizowany jest przez specyficzne mechanizmy:

- **Poufność (Confidentiality):** Gwarantuje, że dane są dostępne wyłącznie dla autoryzowanych podmiotów. W bazach danych osiąga się to poprzez ścisłą kontrolę dostępu (np. RBAC), szyfrowanie danych w spoczynku (TDE - *Transparent Data Encryption*), maskowanie danych wrażliwych oraz szyfrowanie połączeń sieciowych (TLS).

- **Integralność (Integrity):** Zapewnia spójność i poprawność danych, chroniąc je przed nieautoryzowaną modyfikacją. Silniki bazodanowe realizują ten postulat natywnie poprzez właściwości ACID (szczególnie *Consistency*), więzy integralności (klucze obce, ograniczenia *CHECK*), a z punktu widzenia bezpieczeństwa – poprzez audytowanie DML (Data Manipulation Language) i mechanizmy wykrywania anomalii.
- **Dostępność (Availability):** Oznacza pewność, że system odpowie na żądanie w akceptowalnym czasie. Zagrożeniem dla dostępności są awarie sprzętowe oraz ataki DoS. Obroną na poziomie bazy danych są klastry wysokiej dostępności (HA), replikacja (np. *Master-Slave*, *Multi-Master*), partycjonowanie oraz odpowiednio zaprojektowane mechanizmy Disaster Recovery.

Izolacja sieciowa i architektura chmurowa (VPC)

Nawet najlepiej skonfigurowany system uprawnień bazodanowych traci na znaczeniu, jeśli instancja wystawiona jest bezpośrednio do publicznej sieci Internet. Standardem inżynierskim w nowoczesnych wdrożeniach (zarówno on-premise, jak i cloud) jest głęboka separacja sieciowa.

W środowiskach chmurowych (np. AWS, Azure, GCP) bazy danych umieszcza się w dedykowanych chmurach prywatnych (VPC - *Virtual Private Cloud*). Dobrą praktyką jest wdrożenie architektury wielowarstwowej:

1. **Warstwa publiczna (Public Subnet):** Zawiera load balancery i serwery webowe (np. Nginx), do których dostęp ma użytkownik końcowy.
2. **Warstwa aplikacji (Private Subnet):** Środowisko uruchomieniowe backendu (np. Node.js, Spring Boot).
3. **Warstwa danych (Database Subnet):** Najgłębiej ukryta podsieć prywatna bez routingu do Internetu (brak *Internet Gateway*). Dostęp do niej ma wyłącznie warstwa aplikacji poprzez ściśle określone reguły *Security Groups* lub *Network ACLs* (dopuszczające ruch tylko na konkretnym porcie, np. 5432 dla PostgreSQL, i tylko ze znanych adresów IP warstwy aplikacyjnej).

Takie podejście drastycznie zmniejsza powierzchnię ataku (ang. *attack surface*), eliminując ryzyko bezpośrednich ataków typu *brute-force* na porty bazy danych z zewnątrz.

Konteneryzacja a bezpieczeństwo bazy

Uruchamianie baz danych w kontenerach (np. Docker, Kubernetes) stało się powszechne, jednak niesie ze sobą specyficzne wektory zagrożeń. Kontenery dzielą jądro (kernel) z systemem hosta, co oznacza, że kompromitacja bazy danych może prowadzić do ucieczki z kontenera (ang. *container breakout*).

Aby zminimalizować to ryzyko, stosuje się następujące praktyki:

- **Brak uprawnień roota:** Procesy DBMS (np. `postgres` lub `mysqld`) nigdy nie powinny działać jako użytkownik `root` wewnątrz kontenera. Standardowe obrazy często wymagają jawnego zdefiniowania dyrektywy `USER` w pliku `Dockerfile`.
- **Ograniczenia zasobów (Cgroups):** Podatność silnika bazy na ataki typu *Denial of Service* można złagodzić na poziomie infrastruktury, nakładając twarde limity na zużycie CPU i pamięci RAM przez dany kontener. Zapobiega to sytuacji, w której przeciążona baza kładzie cały serwer hosta (*Resource Exhaustion*).
- **Read-Only Root Filesystem:** Kontener bazy danych powinien mieć zamontowany system plików w trybie tylko do odczytu, z wyjątkiem ściśle zdefiniowanych wolumenów (ang. *volumes*) przeznaczonych na pliki danych (np. `/var/lib/postgresql/data`). Uniemożliwia to atakującemu wstrzyknięcie złośliwych skryptów do katalogów systemowych po udanym wykorzystaniu luki aplikacyjnej.

2.7.2 Ataki na bazy danych i luki w zabezpieczeniach

Ponieważ bazy danych przechowują najbardziej krytyczne informacje organizacji, stanowią główny cel cyberataków. Według zestawień takich jak OWASP Top 10, podatności związane ze wstrzykiwaniem kodu (*Injection*) oraz błędną konfiguracją od lat utrzymują się w czołówce zagrożeń. Zrozumienie mechaniki tych ataków jest kluczowe do zaprojektowania skutecznych mechanizmów obronnych w warstwie aplikacyjnej i bazodanowej.

Wstrzykiwanie kodu SQL (SQL Injection)

SQL Injection (SQLi) to podatność polegająca na możliwości modyfikacji zapytania SQL poprzez nieodpowiednio zwalidowane dane wejściowe. Pozwala to atakującemu na wykonanie nieautoryzowanych operacji na bazie danych. Klasyczny przykład podatności (język PHP):

```
$username = $_POST['username'];
$query = "SELECT * FROM users WHERE username = '$username'";
$db->execute($query);
```

Jeśli atakujący jako parametr username przekaże ciąg ' OR '1'='1, zapytanie przyjmie postać:

```
SELECT * FROM users WHERE username = '' OR '1'='1';
```

Ponieważ warunek '1'='1' jest zawsze prawdziwy, silnik bazy danych zwróci wszystkie rekordy z tabeli users, omijając mechanizm uwierzytelniania.

Rodzaje ataków SQLi:

1. **In-Band SQLi (Classic):** Atakujący używa tego samego kanału komunikacyjnego do przeprowadzenia ataku i zebrania wyników. Najpopularniejsze to *Error-based SQLi* (wymuszanie błędów silnika w celu wycieku metadanych, np. struktury tabel) oraz *Union-based SQLi* (użycie operatora UNION do dołączenia złośliwych zapytań do oryginalnego).
2. **Inferential (Blind) SQLi:** Występuje, gdy aplikacja nie zwraca komunikatów o błędach ani bezpośrednich wyników zapytań. Atakujący musi wnioskować o strukturze danych na podstawie zachowania aplikacji. * **Boolean-based:** Wysyłanie zapytań warunkowych i obserwowanie, czy strona ładuje się inaczej. * **Time-based:** Zmuszanie bazy do wstrzymania działania (np. funkcjami pg_sleep() w PostgreSQL lub WAITFOR DELAY w SQL Server), aby potwierdzić obecność podatności.
3. **Out-of-Band SQLi:** Wykorzystywane rzadziej, opiera się na wymuszeniu przez silnik bazy danych żądania HTTP lub DNS do zewnętrznego serwera kontrolowanego przez atakującego.

Mitygacja: Podstawową i najskuteczniejszą metodą obrony przed SQLi jest korzystanie z **zapytań parametryzowanych (Prepared Statements)** lub nowoczesnych systemów ORM (Object-Relational Mapping), które oddzielają składnię zapytań od danych wejściowych.

Ataki NoSQL Injection

Wraz ze wzrostem popularności nierelacyjnych baz danych (np. MongoDB, CouchDB), pojawił się mit o ich całkowitej odporności na wstrzykiwanie kodu. Choć nie używają one tradycyjnego dialektu SQL, aplikacje z nich korzystające nadal mogą być podatne na modyfikację logiki zapytań.

Ataki te często bazują na wstrzykiwaniu operatorów bazy danych do zapytań, które aplikacja interpretuje jako obiekty JSON.

Przykład dla MongoDB: Aplikacja Node.js oczekuje poświadczeń przekazanych w formacie JSON:

```
db.collection('users').find({
  username: req.body.username,
  password: req.body.password
});
```

Atakujący wysyła zmodyfikowany payload z użyciem operatora logicznego \$ne (Not Equal):

```
{
  "username": {"$ne": null},
  "password": {"$ne": null}
}
```

W rezultacie silnik MongoDB wykonuje zapytanie szukające użytkownika, którego login i hasło *nie są nulle*. Zapytanie zwraca pierwszy pasujący rekord (często administratora), umożliwiając logowanie bez znajomości hasła.

Przepełnienie bufora (Buffer Overflow) w DBMS

Silniki baz danych (np. MySQL, PostgreSQL, Oracle) to skomplikowane aplikacje napisane najczęściej w językach C lub C++, co oznacza, że są potencjalnie narażone na błędy zarządzania pamięcią.

Buffer Overflow w kontekście baz danych występuje, gdy atakujący przesyła nieproporcjonalnie duży ciąg danych do bufora o stałej wielkości (np. w parametrze funkcji wbudowanej, mechanizmie autoryzacyjnym lub procedurze składowanej). Gdy dane wykraczają poza zaalokowany obszar pamięci, mogą nadpisać sąsiadujące obszary, w tym wskaźnik powrotu instrukcji (Instruction Pointer).

Może to prowadzić do: 1. **Awarji silnika bazy danych (Crash)** – przerywając ciągłość działania. 2. **Wykonania dowolnego kodu (RCE - Remote Code Execution)** – uruchomienia shellcode'u na serwerze z uprawnieniami procesu bazy danych, co stanowi bezpośrednie wejście do kompromitacji hosta (powiązane z ryzykiem opisanym w rozdziale o konteneryzacji).

Odmowa Usługi (DoS) na poziomie bazy danych

Ataki Denial of Service na bazy danych różnią się od klasycznych ataków sieciowych. Mają one charakter aplikacyjny i celują w wyczerpanie zasobów sprzętowych serwera (CPU, RAM, I/O) lub limitów samego oprogramowania.

- **Wyczerpanie puli połączeń (Connection Exhaustion):** Każdy DBMS ma zdefiniowany limit maksymalnej liczby jednoczesnych połączeń (np. parametr `max_connections` w PostgreSQL). Atakujący, często wykorzystując luki w warstwie aplikacji, może zainicjować tysiące zawieszonych lub bardzo powolnych sesji bazy, uniemożliwiając logowanie legalnym użytkownikom.
- **Asymetryczne obciążenie (Query-based DoS):** Atakujący wstrzykuje i uruchamia zapytania, które są syntaktycznie poprawne, ale drastycznie nieoptymalne kosztowo. Przykładem może być wymuszenie iloczynu kartezjańskiego (CROSS JOIN) na bardzo dużych tabelach bez odpowiednich indeksów, co może zablokować procesor serwera na kilka minut i doprowadzić do blokady całej aplikacji (Deadlock/Timeouts).

Automatyzacja ataków: Narzędzia audytowe

W procesach testów penetracyjnych baz danych, analitycy bezpieczeństwa korzystają ze zautomatyzowanych frameworków. Najpopularniejszym z nich jest **sqlmap** – potężne narzędzie open-source automatyzujące proces wykrywania podatności SQLi i przejmowania kontroli nad serwerami bazodanowymi.

W typowym scenariuszu audytu, uruchomienie polecenia:

```
sqlmap -u "http://podatna-aplikacja.local/item.php?id=1" --dbs
```

pozwoli narzędziu na wykrycie typu wstrzyknięcia (np. czasowe lub błędowe), zidentyfikowanie używanego silnika DBMS oraz – w razie sukcesu – wyliczenie wszystkich baz danych dostępnych na serwerze (enumeracja instancji).

2.7.3 Zarządzanie tożsamością, uwierzytelnianie i autoryzacja

Po zabezpieczeniu infrastruktury sieciowej oraz aplikacji przed atakami typu Injection, kolejną linią obrony jest ścisła kontrola dostępu do samych danych. W architekturze systemów bazodanowych proces ten opiera się na trzech filarach: zarządzaniu tożsamością (Identity Management), uwierzytelnianiu (Authentication) oraz autoryzacji (Authorization). Prawidłowa implementacja tych mechanizmów gwarantuje, że zasada najmniejszego uprzywilejowania (PoLP - Principle of Least Privilege) może być skutecznie egzekwowana w całym systemie.

Uwierzytelnianie w bazach danych

Proces uwierzytelniania weryfikuje, czy podmiot (użytkownik końcowy, administrator lub usługa aplikacyjna) łączący się z bazą danych jest tym, za kogo się podaje. Współczesne silniki relacyjne (np. PostgreSQL, Oracle) oraz nierelacyjne odeszły od prostego przesyłania haseł tekstem jawnym (plaintext), wprowadzając bardziej zaawansowane mechanizmy:

- **Kryptograficzne protokoły uwierzytelniania:** Zamiast przestarzałych algorytmów (np. MD5), nowoczesne wdrożenia wykorzystują mechanizmy typu SCRAM-SHA-256 (Salted Challenge Response Authentication Mechanism). SCRAM zapobiega atakom typu *replay attack* oraz minimalizuje skutki ewentualnej kradzieży skrótów z serwera, ponieważ hasło nigdy nie jest przesyłane w otwartej formie.

- **Integracja z dostawcami tożsamości (IdP):** W środowiskach korporacyjnych unika się tworzenia lokalnych kont wewnątrz bazy (tzw. *database-native accounts*) dla użytkowników fizycznych. Zamiast tego silniki DBMS integruje się z centralnymi usługami katalogowymi (LDAP, Active Directory) lub wykorzystuje protokół Kerberos (np. poprzez GSSAPI). Pozwala to na centralne zarządzanie cyklem życia konta i automatyczne odbieranie dostępów w przypadku odejścia pracownika (tzw. *offboarding*).
- **Uwierzelnianie certyfikatu (mTLS - Mutual TLS):** Standard powszechnie stosowany w architekturze mikroserwisowej do komunikacji *machine-to-machine*. Oprócz szyfrowania samego kanału (TLS), serwer bazy danych żąda i weryfikuje certyfikat klienta (X.509) wystawiony przez wewnętrzne, zaufane centrum certyfikacji (CA).

Autoryzacja i modele kontroli dostępu

Po udanym uwierzelnieniu następuje faza autoryzacji, czyli weryfikacji, jakie operacje (SELECT, INSERT, UPDATE, DELETE) i na jakich obiektach (tabele, widoki, procedury) dany podmiot może wykonać.

Kontrola dostępu oparta na rolach (RBAC - Role-Based Access Control)

Model RBAC to absolutny standard w systemach zarządzania bazami danych. Zamiast nadawać uprawnienia (GRANT) bezpośrednio pojedynczym użytkownikom, tworzy się logiczne role odzwierciedlające funkcje biznesowe lub techniczne (np. `app_readonly`, `app_writer`, `dba_admin`). Następnie konta personalne lub serwisowe przypisuje się do tych ról. Taka abstrakcja drastycznie upraszcza audyt i zarządzanie uprawnieniami.

Przykład implementacji RBAC (dialekt PostgreSQL):

```
-- Utworzenie technicznej roli tylko do odczytu
CREATE ROLE app_readonly;
GRANT CONNECT ON DATABASE production TO app_readonly;
GRANT USAGE ON SCHEMA public TO app_readonly;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO app_readonly;

-- Przypisanie użytkownika raportującego do przygotowanej roli
GRANT app_readonly TO report_user;
```

Bezpieczeństwo na poziomie wierszy (Row-Level Security)

Tradycyjny model autoryzacji w bazach danych działa na poziomie całych struktur (np. blokuje lub zezwala na odczyt całej tabeli). Współczesne aplikacje, zwłaszcza budowane w modelu wielodostępnym (multi-tenant), często wymagają znacznie bardziej granularnej kontroli, aby użytkownik jednego klienta nie miał fizycznej możliwości odczytania rekordów należących do innego.

Z pomocą przychodzi mechanizm Row-Level Security (RLS). Pozwala on na nakładanie polityk bezpieczeństwa bezpośrednio na wiersze danych. Silnik bazy automatycznie i w sposób przezroczysty dokleja zdefiniowane warunki do każdego zapytania. Kluczową zaletą RLS jest fakt, że uniemożliwia on ominięcie logiki izolacyjnej w warstwie aplikacji – nawet w przypadku wystąpienia luki SQL Injection.

Przykład definicji RLS (dialekt PostgreSQL):

```
-- Włączenie weryfikacji RLS dla krytycznej tabeli
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;

-- Utworzenie polityki izolującej dane między użytkownikami
CREATE POLICY isolate_tenant_policy ON orders
USING (tenant_id = current_setting('app.current_tenant')::int);
```

Zarządzanie poświadczeniami aplikacji (Secrets Management)

Nawet najbardziej rygorystyczne mechanizmy wewnątrz samej bazy danych zawiodą, jeśli poświadczenia do niej (tzw. *connection strings*) zostaną umieszczone na stałe (hardcoded) w kodzie źródłowym i wypchnięte do repozytorium kodu.

Zgodnie z dobrymi praktykami nurtu DevSecOps, środowiska produkcyjne powinny wykorzystywać zewnętrzne systemy zarządzania sekretami (np. HashiCorp Vault, AWS Secrets Manager). Narzędzia te pozwalają na dynamiczne generowanie tymczasowych, krótkożyjących poświadczeń (tzw. *dynamic secrets*). W takim scenariuszu aplikacja uwierzytelnia się do Vaulta i otrzymuje login oraz hasło do bazy danych, które wygasają np. po godzinie. Po tym czasie usługa zarządzająca automatycznie usuwa (REVOKE) te poświadczenia z silnika bazy, co radykalnie zmniejsza okno potencjalnego ataku w przypadku ich wycieku.

2.7.4 Kryptografia w bazach danych

Nawet najbardziej rygorystyczne mechanizmy kontroli dostępu i izolacji sieciowej mogą zawieść w obliczu zaawansowanych ataków (tzw. APT – Advanced Persistent Threats) lub błędów ludzkich. W myśl zasady obrony w głąb (Defense in Depth), ostatnią linią ochrony informacji jest kryptografia. Jej implementacja w środowiskach bazodanowych dzieli się na trzy główne obszary: ochronę danych w spoczynku, w tranzycie oraz w użyciu.

Szyfrowanie danych w spoczynku (Data at Rest)

Szyfrowanie w spoczynku ma na celu zabezpieczenie fizycznych nośników danych przed nieautoryzowanym odczytem, na przykład w przypadku kradzieży serwera, dysku twardego lub nieautoryzowanego skopiowania plików kopii zapasowych.

Przemysłowym standardem w tym zakresie jest mechanizm **TDE (Transparent Data Encryption)**, natywnie wspierany przez silniki takie jak Microsoft SQL Server czy Oracle (dla PostgreSQL stosuje się często szyfrowanie na poziomie systemu plików, np. LUKS, lub dedykowane rozszerzenia). TDE działa na poziomie warstwy wejścia/wyjścia (I/O) silnika bazy danych:

- **Przezroczystość:** Aplikacja kliencka nie musi być modyfikowana. Dane są szyfrowane tuż przed zapisem na dysk i deszyfrowane w momencie wczytywania ich do pamięci operacyjnej (RAM) serwera.
- **Architektura kluczy:** Bezpieczeństwo TDE opiera się na hierarchii kluczy kryptograficznych. Dane właściwe szyfrowane są kluczem symetrycznym DEK (Data Encryption Key). Sam DEK jest z kolei chroniony (szyfrowany) asymetrycznym kluczem nadrzędnym MEK (Master Encryption Key), który powinien być przechowywany poza samym serwerem bazy danych – najczęściej w sprzętowych modułach bezpieczeństwa (HSM) lub chmurowych usługach KMS (Key Management Service).

Szyfrowanie danych w tranzycie (Data in Transit)

Przesyłanie danych między warstwą aplikacji a silnikiem bazy danych w postaci jawnej (plaintext) naraża system na ataki typu Man-in-the-Middle (MitM) oraz podsłuchiwanie pakietów (sniffing).

Aby temu zapobiec, wszystkie połączenia z bazą danych muszą być obligatoryjnie szyfrowane za pomocą protokołu **TLS (Transport Layer Security)**. Wymuszenie bezpiecznego kanału realizuje się zazwyczaj poprzez odpowiednią konfigurację parametrów połączenia.

Przykład wymuszenia TLS w konfiguracji PostgreSQL (`postgresql.conf`):

```
# Włączenie obsługi protokołu TLS
ssl = on
ssl_cert_file = '/etc/ssl/certs/db-server.crt'
ssl_key_file = '/etc/ssl/private/db-server.key'
# Odrzucanie połączeń używających przestarzałych, podatnych wersji protokołu
ssl_min_protocol_version = 'TLSv1.2'
```

Po stronie klienta, ciąg połączeniowy (connection string) powinien zawierać dyrektywę bezwzględnie wymagającą weryfikacji certyfikatu (np. parametr `sslmode=verify-full`).

Szyfrowanie na poziomie klienta (Client-Side Encryption)

Mechanizm TDE chroni przed fizyczną kradzieżą nośników, ale nie zabezpiecza przed administratorem bazy danych (DBA) lub atakiem SQL Injection, ponieważ w momencie działania zapytania dane w pamięci serwera są w pełni odszyfrowane.

Odpowiedzią na ten problem jest szyfrowanie na poziomie aplikacji (lub kolumn w nowszych silnikach, np. funkcja *Always Encrypted* w SQL Server). W tym paradygmacie dane są szyfrowane w warstwie aplikacji (np. backendzie w Javie lub C#) jeszcze przed wysłaniem ich do bazy.

Silnik bazy danych operuje wyłącznie na kryptogramach (ciphertext) i nigdy nie ma dostępu do kluczy deszyfrujących, co zapewnia najwyższy poziom poufności dla najbardziej wrażliwych kolumn (np. numerów PESEL, danych kart kredytowych). Kosztem takiego rozwiązania jest utrata możliwości wykonywania zaawansowanych operacji analitycznych (takich jak sumowanie, sortowanie czy wyszukiwanie z użyciem operatora LIKE) bezpośrednio po stronie DBMS.

Dynamiczne maskowanie danych i funkcje skrótu

Uzupełnieniem kryptografii jest technika **DDM (Dynamic Data Masking)**. Służy ona do ochrony wrażliwych danych w czasie rzeczywistym przed użytkownikami, którzy nie posiadają odpowiednich poświadczeń (np. analitykami biznesowymi lub programistami korzystającymi z kopii produkcyjnej). DDM maskuje wyniki zapytań (np. zwracając ciąg ****-****-****-1234 zamiast pełnego numeru karty), podczas gdy same dane na dysku pozostają nienaruszone.

Ponadto, wracając do zagadnień z zakresu uwierzytelniania, należy pamiętać o kryptografii jednokierunkowej. Wrażliwe poświadczenia (takie jak hasła użytkowników aplikacji) nigdy nie mogą być przechowywane w formie szyfrowalnej (dwukierunkowej). Zamiast tego muszą być poddawane procesowi solenia (salting) i hashowania z wykorzystaniem funkcji spowalniających (Key Derivation Functions), takich jak **Argon2**, **bcrypt** lub **PBKDF2**, co skutecznie uniemożliwia ataki słownikowe i wykorzystanie tęczyowych tablic (rainbow tables).

2.7.5 Audyt, logowanie i wykrywanie anomalii

Nawet najsilniejsze mechanizmy prewencyjne, takie jak kontrola dostępu czy szyfrowanie, nie gwarantują stuprocentowego bezpieczeństwa. W przypadku udanego ataku lub błędu wewnętrznego (tzw. zagrożenia *Insider Threat*), kluczowe staje się odtworzenie przebiegu zdarzeń. Temu celowi służą mechanizmy audytu i logowania, które stanowią fundament dla systemów wykrywania anomalii oraz zapewniają niezaprzeczalność (ang. *Non-repudiation*) operacji wykonywanych na danych.

Znaczenie audytu i rodzaje logów bazodanowych

Audytowanie bazy danych to proces polegający na monitorowaniu i rejestrowaniu akcji użytkowników oraz procesów systemowych. Nie należy mylić go z klasycznymi logami transakcyjnymi (np. WAL w PostgreSQL czy Redo Logs w Oracle), które służą do odtwarzania bazy po awarii. Audyt bezpieczeństwa skupia się na tym **kto**, **kiedy**, **skąd** i **jaka** operację wykonał.

Większość nowoczesnych silników DBMS pozwala na granularną konfigurację logowania. Rejestrowane zdarzenia można podzielić na kilka kategorii: * **Logi uwierzytelniania**: Rejestrują udane i nieudane próby logowania (np. błędy *Access Denied*), co pozwala na szybkie wykrycie ataków typu brute-force. * **Logi DDL (Data Definition Language)**: Śledzą zmiany w strukturze bazy (tworzenie i usuwanie tabel, modyfikacje uprawnień). * **Logi DML (Data Manipulation Language)**: Najbardziej kosztowne wydajnościowo, ale niezbędne w systemach o wysokim rygorze bezpieczeństwa (np. bankowość, opieka zdrowotna). Pozwalają na śledzenie zapytań SELECT, UPDATE czy DELETE na wrażliwych tabelach.

Przykładowo, natywne logowanie w PostgreSQL jest często niewystarczające dla zaawansowanego audytu. W środowiskach produkcyjnych powszechnie stosuje się rozszerzenie **pgAudit** (PostgreSQL Audit Extension), które dostarcza szczegółowe logowanie sesji i obiektów przy użyciu standardowych mechanizmów logowania wbudowanych w silnik.

Centralizacja logów i systemy SIEM

Przechowywanie logów audytowych wyłącznie na tym samym serwerze, na którym działa baza danych, jest poważnym błędem architektonicznym. W przypadku udanej kompromitacji hosta (np. poprzez ucieczkę z kontenera opisaną w rozdziale 1), atakujący w pierwszej kolejności zacierza ślady, modyfikując lub usuwając pliki logów.

Dlatego standardem rynkowym jest natychmiastowy eksport logów w czasie rzeczywistym do niezależnych systemów centralnych. Wykorzystuje się do tego architekturę SIEM (ang. *Security Information and Event Manage-*

ment), z wykorzystaniem stosów technologicznych takich jak **ELK** (Elasticsearch, Logstash, Kibana) lub **Splunk**. Systemy te agregują logi nie tylko z baz danych, ale również z warstwy aplikacji, firewalli i systemów operacyjnych, pozwalając na korelację zdarzeń z różnych źródeł w celu identyfikacji złożonych ataków.

Database Activity Monitoring (DAM) i wykrywanie anomalii

Tradycyjna analiza logów jest procesem reaktywnym (po fakcie). Aby przejść na model proaktywny, stosuje się systemy DAM (ang. *Database Activity Monitoring*), które monitorują ruch do bazy danych z zewnątrz (np. analizując ruch sieciowy (tzw. *packet sniffing*) lub podpinając się pod pule pamięci silnika). Dzięki temu system DAM jest odseparowany od samej bazy, co zapobiega modyfikacji jego działania nawet po przejęciu uprawnień administratora bazy (DBA).

Kluczową funkcją nowoczesnych systemów z tej kategorii jest **wykrywanie anomalii w oparciu o analizę behawioralną (UEBA)**. Zamiast opierać się wyłącznie na statycznych sygnaturach zagrożeń, systemy te wykorzystują algorytmy uczenia maszynowego do modelowania “normalnego” zachowania aplikacji i użytkowników.

System wykryje anomalię i podniesie alarm (lub zablokuje zapytanie), gdy np.: * Aplikacja webowa, która zazwyczaj pobiera pojedyncze rekordy w zapytaniach SELECT, nagle próbuje wykonać zrzut (ang. *dump*) tysięcy wierszy z tabeli klientów – co może świadczyć o udanym wstrzyknięciu kodu (SQLi z rozdziału 2) i próbie ekstrakcji danych. * Użytkownik o uprawnieniach administracyjnych loguje się do bazy z nietypowej geolokalizacji lub poza standardowymi godzinami pracy. * Gwałtownie rośnie liczba odrzucanych połączeń lub błędów składniowych SQL z określonego adresu IP wewnątrz sieci.

2.7.6 Bezpieczeństwo kopii zapasowych i Disaster Recovery

Systemy prewencyjne i detekcyjne nie zawsze są w stanie zatrzymać zaawansowane ataki, takie jak ransomware, czy też zapobiec fizycznym awariom infrastruktury. W takich scenariuszach ostateczną linią obrony stają się mechanizmy odtwarzania danych po awarii. Bezpieczeństwo samych kopii zapasowych jest równie krytyczne co bezpieczeństwo głównej instancji bazy danych – skompromitowany backup często ostatecznie przekreśla szanse na powrót organizacji do normalnego funkcjonowania operacyjnego.

Zasada 3-2-1 i niezmienność kopii (Immutable Backups)

Klasyczna reguła 3-2-1 (trzy kopie, na dwóch różnych nośnikach, z czego jedna w innej lokalizacji fizycznej lub geograficznej) pozostaje fundamentem ochrony danych. Jednakże, w dobie zaawansowanych ataków kryptograficznych, wymaga ona uzupełnienia o koncepcję niezmienności (ang. *Immutability*).

Współczesne złośliwe oprogramowanie często priorytetyzuje poszukiwanie dysków sieciowych oraz zasobów chmurowych z kopiami zapasowymi, aby je usunąć lub zaszyfrować przed zaatakowaniem głównego klastra bazy danych. Odpowiedzią inżynierską na to zagrożenie jest technologia zapisu WORM (ang. *Write Once, Read Many*). W nowoczesnej architekturze chmurowej (np. za pomocą mechanizmu AWS S3 Object Lock) gwarantuje ona, że zapisany plik backupu nie może zostać w żaden sposób zmodyfikowany ani usunięty przez zdefiniowany okres retencji. Ochrona ta działa nawet w przypadku całkowitej kompromitacji konta z najwyższymi uprawnieniami (ang. *root account compromise*).

Szyfrowanie archiwów i zarządzanie kluczami

Ponieważ logiczne i fizyczne kopie zapasowe opuszczają bezpieczne środowisko serwerowni lub głęboko izolowanej podsieci (VPC, o której mowa w rozdziale 1), absolutnym wymogiem jest ich szyfrowanie w spoczynku (ang. *Data at Rest Encryption*). Wykonanie zwykłego zrzutu bazy w postaci czystego tekstu i przesłanie go na zewnątrz to drastyczne ryzyko wycieku, uderzające bezpośrednio w postulat poufności.

W nowoczesnych wdrożeniach archiwizacji stosuje się dedykowane systemy KMS (ang. *Key Management Service*). Klucze szyfrujące używane do ochrony backupów powinny być ściśle odseparowane od infrastruktury samego klastra bazodanowego. Dodatkowo podczas procesu odtwarzania konieczna jest weryfikacja integralności archiwów przy użyciu funkcji skrótu (np. SHA-256), co pozwala upewnić się, że paczka danych nie uległa uszkodzeniu na etapie przesyłania.

Disaster Recovery (DR) i wskaźniki RTO/RPO

Kopia zapasowa to jedynie nośnik danych; z perspektywy ciągłości biznesowej (ang. *Business Continuity*) liczy się sprawdzony proces ich przywracania. Architektura Disaster Recovery dla baz danych opiera się na dwóch krytycznych metrykach:

- **RPO (Recovery Point Objective):** Maksymalny akceptowalny czas, z którego dane mogą zostać bezpowrotnie utracone. Minimalizację tego wskaźnika do okolic zera osiąga się poprzez ciągłą strumieniową archiwizację logów transakcyjnych (np. mechanizm *WAL archiving* w PostgreSQL).
- **RTO (Recovery Time Objective):** Czas niezbędny na przywrócenie systemów bazodanowych do pełnego działania operacyjnego po awarii.

Dla systemów o rygorystycznym RTO (wymagających dostępności mierzonej w sekundach) tradycyjne odtwarzanie z plików jest niewystarczające. Stosuje się wówczas architekturę klastrów rozproszonych w modelach *Active-Passive* (ciągła, asynchroniczna replikacja do zapasowego centrum danych) lub *Active-Active* (synchroniczny zapis w wielu lokalizacjach, wymagający specjalnego rozwiązywania problemów ze spójnością i opóźnieniami sieciowymi, tzw. *split-brain*).

Niezależnie od stopnia skomplikowania architektury chmurowej, każda polityka Disaster Recovery pozostaje martwym dokumentem, dopóki nie zostanie poddana rygorystycznym, regularnym i udokumentowanym testom odtworzeniowym (*Disaster Recovery Drills*) w odizolowanym środowisku.

2.8 Kopie zapasowe i odzyskiwanie danych

Autorzy

Norbert Antonovitch, Michał Bednarczyk, Jan Balazs de Borbatviz

2.8.1 Wstęp

Kopie zapasowe w PostgreSQL można podzielić na dwa główne nurty: kopie logiczne wykonywane narzędziami `pg_dump` i `pg_dumpall` oraz kopie fizyczne całego klastra wykonywane m.in. przez `pg_basebackup` wraz z archiwizacją WAL dla odzyskiwania punktowego PITR (Point-in-Time Recovery). Kopie logiczne są wygodne do odtwarzania pojedynczych obiektów i pojedynczych baz, natomiast kopie fizyczne są podstawą pełnego odtworzenia instancji, tablespaces i odzyskiwania po awariach.

2.8.2 Tworzenie kopii zapasowej całego systemu PostgreSQL - mechanizmy wbudowane

Najprostszym wbudowanym sposobem wykonania kopii całego klastra jest `pg_dumpall`, które eksportuje wszystkie bazy w klastrze oraz obiekty globalne wspólne dla całej instancji, takie jak role i przestrzenie tabel. Taki zrzut ma postać tekstowego skryptu SQL i odtwarza się go zwykle przez `psql`, ale metoda ta jest logiczna, więc nie daje możliwości odzyskiwania punktowego przez WAL.

Drugim, ważniejszym z punktu widzenia odporności na awarie mechanizmem jest `pg_basebackup`, które tworzy fizyczną kopię działającego klastra bez blokowania normalnej pracy klientów. Narzędzie to wykonuje kopię całego klastra, a nie pojedynczych baz czy tabel, może pracować w formacie katalogowym lub `tar` i może dołączać wymagane pliki WAL metodą `stream`, `fetch` albo `none`. Dokumentacja podkreśla też, że taka kopia może służyć zarówno do PITR, jak i do zbudowania serwera standby.

Aby pełna kopia fizyczna była użyteczna w scenariuszu odtwarzania do wybranego momentu, należy włączyć archiwizację WAL przez ustawienie co najmniej `wal_level = replica`, `archive_mode = on` oraz poprawnego `archive_command` albo `archive_library`. PostgreSQL zapisuje każdą zmianę w logu WAL, a po odtworzeniu kopii bazowej można odtworzyć kolejne segmenty WAL, aby dojść do stanu bieżącego lub do wskazanego punktu w czasie.

Przykładowe polecenia:

```
# logiczna kopia całego klastra
pg_dumpall -U postgres --clean --file=cluster.sql
```

(continues on next page)

(continued from previous page)

```
# fizyczna kopia całego klastra z dołączeniem WAL
pg_basebackup -h dbserver -D /backup/base -Fp -X stream -P
```

W praktyce do dokumentacji laboratoryjnej warto zaznaczyć, że `pg_dumpall` lepiej nadaje się do migracji i prostych kopii administracyjnych, a `pg_basebackup` z archiwizacją WAL do scenariuszy disaster recovery.

2.8.3 Tworzenie kopii zapasowych poszczególnych baz danych - mechanizmy wbudowane

Do wykonania kopii pojedynczej bazy służy `pg_dump`, które tworzy spójny zrzut nawet wtedy, gdy baza jest równocześnie używana przez innych użytkowników, i nie blokuje zwykłych operacji odczytu ani zapisu. Narzędzie obsługuje format tekstowy, custom, directory oraz tar, przy czym formaty archiwalne współpracują z `pg_restore` i pozwalają na selektywne odtwarzanie obiektów.

Istotne jest to, że `pg_dump` wykonuje kopię tylko jednej bazy danych. Format custom (`-Fc`) i directory (`-Fd`) są najbardziej elastyczne, bo pozwalają wybierać odtwarzane elementy, zmieniać kolejność odtwarzania, a w części scenariuszy także korzystać z odtwarzania równoległego.

Przykładowe polecenia:

```
# kopia pojedynczej bazy w formacie custom
pg_dump -U postgres -d labdb -Fc -f /backup/labdb.dump

# kopia pojedynczej bazy jako skrypt SQL
pg_dump -U postgres -d labdb -Fp -f /backup/labdb.sql

# odtworzenie archiwum custom
pg_restore -d labdb_restored /backup/labdb.dump
```

Jeżeli celem jest ochrona wybranych schematów lub tabel, `pg_dump` pozwala zawęzić zakres eksportu przez opcje takie jak `-n` dla schematu i `-t` dla tabeli. Trzeba jednak zaznaczyć, że wybiórczy zrzut nie gwarantuje kompletności zależności wszystkich obiektów w nowym, pustym środowisku, jeśli pominięto elementy, od których zależy odtwarzany obiekt.

2.8.4 Odzyskiwanie usuniętych lub uszkodzonych danych

Sposób odzyskiwania zależy od tego, czy problem dotyczy pojedynczego obiektu logicznego, całej bazy, przestrzeni tabel, czy fizycznego uszkodzenia danych. PostgreSQL rozróżnia w praktyce odzyskiwanie logiczne z plików dump oraz odzyskiwanie fizyczne z kopii bazowej i archiwum WAL.

Odtwarzanie tabel i innych obiektów logicznych

Jeżeli wcześniej wykonano kopię `pg_dump` w formacie archiwalnym, można odtworzyć pojedynczą tabelę lub inny obiekt selektywnie przy pomocy `pg_restore`, bez przywracania całej bazy. To jest najprostszy wariant odzyskania przypadkowo usuniętej tabeli, o ile odpowiedni obiekt znajdował się w logicznej kopii zapasowej.

Przykład:

```
pg_restore -d labdb -t public.wyniki /backup/labdb.dump
```

Odtwarzanie usuniętej bazy danych

Usuniętą bazę można najłatwiej odtworzyć z wcześniejszego zrzutu `pg_dump` tej konkretnej bazy albo z `pg_dumpall`, jeżeli wykonywano kopię całego klastra. W przypadku wymogu odtworzenia dokładnie do chwili sprzed usunięcia, potrzebna jest kopia fizyczna oraz archiwum WAL, ponieważ same zrzuty logiczne nie są częścią rozwiązywania continuous archiving i nie wspierają PITR.

Odtwarzanie przestrzeni tabel i pełnego klastra

Przestrzenie tabel są obiektami globalnymi klastra, dlatego ich definicje są uwzględniane przez `pg_dumpall`, a przy kopiach fizycznych `pg_basebackup` zachowuje także ich strukturę i mapowanie. Dokumentacja `pg_basebackup` wskazuje, że przy pracy z tablespaces można użyć mapowania `--tablespace-mapping`, a przy odtwarzaniu trzeba zweryfikować poprawność dowiązań symbolicznych i lokalizacji danych.

Przy fizycznym uszkodzeniu danych, katalogu PGDATA albo tablespaces standardowa procedura obejmuje odtworzenie kopii bazowej, usunięcie starych plików danych, odtworzenie katalogów danych i tablespaces, skonfigurowanie `restore_command`, utworzenie pliku `recovery.signal`, a następnie ponowne uruchomienie serwera w trybie recovery. PostgreSQL podczas odtwarzania odczytuje wymagane segmenty WAL i może dojść do stanu bieżącego albo zatrzymać się na wybranym punkcie odzyskiwania.

PITR i odzyskiwanie stanu sprzed błędu

Najważniejszym mechanizmem odzyskiwania po usunięciu danych operacyjnych jest PITR, czyli przywrócenie systemu do chwili sprzed błędnej operacji, na przykład `DROP TABLE` lub `DROP DATABASE`. W takim scenariuszu odtwarza się kopię bazową, konfiguruje `restore_command`, a następnie ustawia cel odzyskiwania, np. przez czas, nazwany punkt przywracania lub identyfikator transakcji.

To rozwiązanie ma jednak ważne ograniczenie: nie służy do odzyskania pojedynczej tabeli “w miejscu” bez wpływu na resztę systemu, lecz do odtworzenia całego klastra do wcześniejszego stanu. Dlatego w praktyce laboratoryjnej często łączy się oba podejścia: regularne `pg_dump` dla obiektów logicznych i `pg_basebackup` z archiwizacją WAL dla pełnego recovery.

2.8.5 Dedykowane oprogramowanie i skrypty zewnętrzne do automatyzacji

Wbudowane narzędzia PostgreSQL są wystarczające do ręcznego wykonywania kopii, ale w środowisku produkcyjnym często wykorzystuje się oprogramowanie automatyzujące harmonogramy, retencję, walidację i odtwarzanie. Dwa najczęściej wskazywane rozwiązania to Barman i pgBackRest, przy czym dostępna dokumentacja Barman bezpośrednio opisuje obsługę pełnych i przyrostowych kopii, archiwizacji WAL oraz automatycznego uruchamiania backupów.

Barman wykonuje kopie całego serwera PostgreSQL i wymaga poprawnej archiwizacji WAL, oferując różne metody kopii, w tym streaming przez `pg_basebackup`, `rsync` oraz backupy snapshotowe w chmurze. Dokumentacja podaje też wsparcie dla backupów przyrostowych, limitowania pasma, kompresji i pracy z harmonogramem przez zewnętrzne mechanizmy systemowe, np. `cron`.

Przykładowe zastosowania narzędzi zewnętrznych:

- **Barman:** centralne repozytorium backupów, archiwizacja WAL, backupy pełne i przyrostowe, odzyskiwanie całych instancji.
- **pgBackRest:** popularne narzędzie do wydajnych backupów i retencji, często używane w środowiskach HA oraz większych instalacjach PostgreSQL.
- **Własne skrypty Bash/PowerShell:** wywołanie `pg_dump`, `pg_dumpall` lub `pg_basebackup`, kompresja plików, rotacja katalogów, logowanie i wysyłka wyników przez e-mail lub do systemu monitoringu.

Przykładowy prosty skrypt automatyzujący kopię jednej bazy:

```
#!/bin/bash
DATA=$(date +%F_%H-%M)
KATALOG=/backup/postgres
BAZA=labdb

mkdir -p "$KATALOG"
pg_dump -U postgres -d "$BAZA" -Fc -f "$KATALOG/${BAZA}_${DATA}.dump"
find "$KATALOG" -type f -name "${BAZA}_*.dump" -mtime +7 -delete
```

Taki skrypt jest prosty, ale nie zapewnia pełnego disaster recovery, bo nie obejmuje archiwizacji WAL i nie chroni całego klastra. Z tego powodu dla systemów krytycznych bardziej właściwe są narzędzia klasy Barman lub pgBackRest, które porządkują pełny proces backupu, retencji, testów i odtwarzania.

2.8.6 Podsumowanie

W rozdziale warto wyraźnie rozróżnić kopie logiczne i fizyczne, bo odpowiadają one na inne potrzeby eksploatacyjne. `pg_dump` i `pg_dumpall` są najlepsze do prostych eksportów i selektywnego odtwarzania, natomiast `pg_basebackup` z archiwizacją WAL jest podstawą odzyskiwania po poważnej awarii i realizacji PITR.

PLANOWANIE BAZ DANYCH I TWORZENIE DOKUMENTACJI

Autorzy

Norbert Antonovitch, Michał Bednarczyk, Jan Balazs de Borbatviz

3.1 Wybór zagadnienia i identyfikacja danych

Tematem projektu jest podstawowa baza danych obsługująca sklep internetowy ze sprzętem elektronicznym. Baza danych zawiera następujące grupy danych:

- **Klienci:** unikalny identyfikator, imię, nazwisko, adres e-mail, numer telefonu oraz podstawowe dane adresowe do wysyłki: ulica, miasto, kod pocztowy.
- **Produkty:** unikalny identyfikator, nazwa sprzętu, cena jednostkowa oraz przypisana kategoria.
- **Kategorie:** identyfikator kategorii oraz jej nazwa, na przykład Laptopy lub Smartfony.
- **Zamówienia:** identyfikator zamówienia, data złożenia, status, na przykład nowe lub zrealizowane, oraz powiązanie z konkretnym klientem.
- **Szczegóły zamówienia:** pozycje przypisane do konkretnego zamówienia, określające jaki produkt został kupiony, w jakiej ilości oraz po jakiej cenie.

3.2 Opis procesów i towarzyszących im danych

1. **Zarządzanie produktami:** sklep oferuje asortyment przypisany do konkretnych kategorii, na przykład laptopy i smartfony. Każdy produkt ma przypisaną nazwę, cenę oraz identyfikator producenta.
2. **Zarządzanie użytkownikami:** klienci rejestrują się w systemie, podając podstawowe dane, takie jak imię, nazwisko i e-mail, oraz pojedynczy główny adres zamieszkania wykorzystywany do wysyłki.
3. **Realizacja zamówień:** zalogowany klient składa zamówienie na wybrane produkty. System tworzy nagłówek zamówienia zawierający datę i status oraz przypisuje do niego poszczególne produkty wraz z ich ilością.

3.3 Prototypowy plik JSON

Poniżej przedstawiono prototypowy plik w formacie JSON, zawierający jeden pełny dokument reprezentujący złożone zamówienie. Pozwala to zweryfikować kompletność gromadzonych i przetwarzanych informacji przed przejściem do modelowania konceptualnego.

```
{
  "id_zamowienia": 1,
  "data_zlozenia": "2026-05-14T10:15:30Z",
  "status": "Nowe",
  "klient": {
    "id_klienta": 4096,
    "imie": "Jan",
    "nazwisko": "Kowalski",
    "email": "student01@example.com",
    "telefon": "123456789",
    "ulica": "Armii Krajowej 78",
    "kod_pocztowy": "58-300",
```

(continues on next page)

(continued from previous page)

```

    "miasto": "Walbrzych"
  },
  "pozycje": [
    {
      "id_produktu": 10543,
      "nazwa": "Laptop ASUS",
      "kategoria": "Laptopy",
      "ilosc": 1,
      "cena_historyczna": 6299.99
    }
  ]
}

```

3.3.1 Dobór typów danych

Table 1: Mapowanie typów danych dla SQLite i PostgreSQL

Atrybut danych	SQLite	PostgreSQL
Identyfikatory (id_...)	INTEGER	SERIAL lub INTEGER
Napisy krótkie (imie, nazwisko, miasto)	TEXT	VARCHAR(50)
Napisy długie (ulica, email, nazwa)	TEXT	VARCHAR(255)
Stałe ciągi znaków (telefon, kod_pocztowy)	TEXT	VARCHAR(15)
Cena (cena_bazowa, cena_historyczna)	FLOAT	NUMERIC(10,2)
Ilość (ilosc)	INTEGER	SMALLINT
Data i czas (data_zlozenia)	TEXT	TIMESTAMP

3.4 Modelowanie konceptualne i logiczne

3.4.1 Model koncepcyjny

1. Encje mocne

- **Klient:** klucz główny (id_klienta), imię, nazwisko, e-mail, telefon, ulica, miasto, kod pocztowy.
- **Produkt:** klucz główny (id_produktu), nazwa, cena_bazowa.
- **Kategoria:** klucz główny (id_kategorii), nazwa.
- **Zamówienie:** klucz główny (id_zamowienia), data_zlozenia, status.

2. Encje słabe

- **Szczegóły zamówienia:** klucz obcy (id_zamowienia), klucz obcy (id_produktu), ilość, cena_historyczna. Jej identyfikacja opiera się wyłącznie na kluczach obcych pochodzących z encji silnych.

3. Opis związków

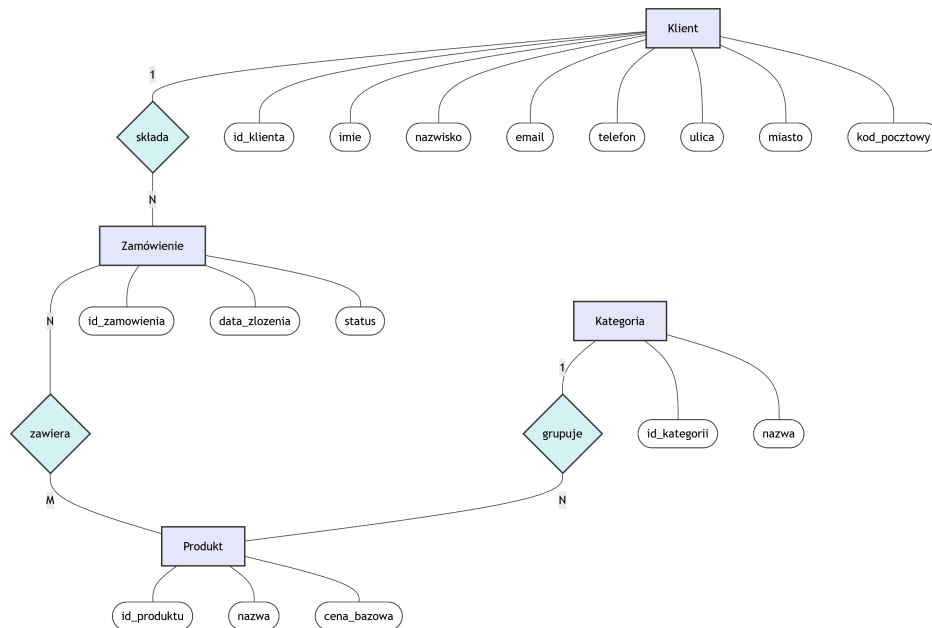
Zdefiniowano następujące relacje biznesowe między encjami:

- **Klient – Zamówienie (1:N):** klient składa zamówienie. Jeden klient może posiadać wiele zamówień, ale jedno konkretne zamówienie jest przypisane dokładnie do jednego klienta.
- **Kategoria – Produkt (1:N):** kategoria grupuje produkty. Kategoria może zawierać wiele produktów, ale dany produkt należy do jednej kategorii.
- **Zamówienie – Produkt (M:N):** zamówienie zawiera produkty. Pojedyncze zamówienie obejmuje wiele produktów, a dany produkt może pojawić się w wielu zamówieniach.

4. Związki niepoprawne

Wyeliminowano bezpośrednią relację między Klientem a Produktem, aby uniknąć pułapki połączeń. Bezpośrednie powiązanie gubi kontekst transakcji, na przykład datę i ilość zakupionego towaru, dlatego poprawna relacja musi zawsze przechodzić przez encję Zamówienie.

3.4.2 Model koncepcyjny – notacja Chena

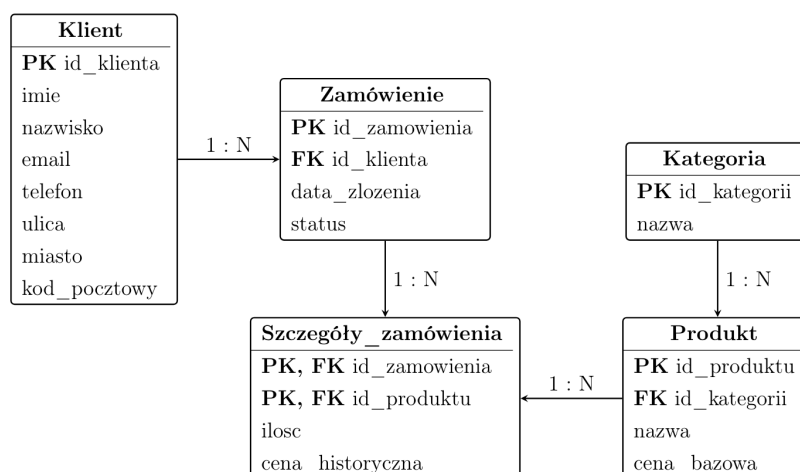


3.4.3 Model logiczny i normalizacja

Główne działania podjęte podczas normalizacji:

1. **I Postać Normalna (1NF):** dane adresowe klienta, takie jak ulica, miasto i kod pocztowy, zostały wyodrębnione jako pojedyncze pola.
2. **II Postać Normalna (2NF):** zlikwidowano relację wiele-do-wielu (N:M) pomiędzy Zamówieniem a Produktem, wstawiając encję asocjacyjną Szczegóły_zamówienia. Jej klucz główny składa się z identyfikatora zamówienia oraz identyfikatora produktu.
3. **III Postać Normalna (3NF):** usunięto zależności przechodnie. Nazwa kategorii została wydzielona do osobnej tabeli Kategorii. W tabeli Produkty przechowywany jest jedynie klucz obcy do kategorii. Ponadto zapewniono zapisywanie historycznej ceny w Szczegółach_zamówienia, aby modyfikacja cennika nie fałszowała archiwalnych rachunków.

3.4.4 Schemat notacji IE po normalizacji



DEFINIOWANIE BAZY DANYCH I WPROWADZANIE DANYCH DO BAZY

Autorzy

Norbert Antonovitch, Michał Bednarczyk, Jan Balazs de Borbatviz

4.1 Definicja bazy danych w wariantach PostgreSQL i SQLite

W projekcie zastosowano dwa warianty definicji schematu bazy danych: dla PostgreSQL oraz dla SQLite. Oba warianty odwzorowują tę samą strukturę logiczną, jednak różnią się doбором typów danych i składnią definicji kluczy głównych oraz obcych.

W wariancie PostgreSQL przyjęto rygorystyczne typowanie danych. Identyfikatory zdefiniowano z użyciem typu SERIAL, pola tekstowe opisano typem VARCHAR o ograniczonej długości, natomiast wartości finansowe zapisano w typie NUMERIC(10,2), zapewniającym odpowiednią precyzję obliczeń.

```
-- Wariant PostgreSQL
CREATE TABLE kategorie (
    id_kategorii SERIAL PRIMARY KEY,
    nazwa VARCHAR(100) NOT NULL
);

CREATE TABLE klienci (
    id_klienta SERIAL PRIMARY KEY,
    imie VARCHAR(50) NOT NULL,
    nazwisko VARCHAR(50) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    telefon VARCHAR(15),
    ulica VARCHAR(255),
    miasto VARCHAR(50),
    kod_pocztowy VARCHAR(15)
);

CREATE TABLE produkty (
    id_produktu SERIAL PRIMARY KEY,
    id_kategorii INTEGER NOT NULL,
    nazwa VARCHAR(255) NOT NULL,
    cena_bazowa NUMERIC(10,2) NOT NULL,
    CONSTRAINT fk_kategoria
        FOREIGN KEY (id_kategorii) REFERENCES kategorie(id_kategorii)
);

CREATE TABLE zamowienia (
    id_zamowienia SERIAL PRIMARY KEY,
    id_klienta INTEGER NOT NULL,
    data_zlozenia TIMESTAMP NOT NULL,
    status VARCHAR(50) NOT NULL,
    CONSTRAINT fk_klient
```

(continues on next page)

(continued from previous page)

```

        FOREIGN KEY (id_klienta) REFERENCES klienci(id_klienta)
    );

CREATE TABLE szczegoly_zamowienia (
    id_zamowienia INTEGER NOT NULL,
    id_produktu INTEGER NOT NULL,
    ilosc SMALLINT NOT NULL CHECK (ilosc > 0),
    cena_historyczna NUMERIC(10,2) NOT NULL,
    PRIMARY KEY (id_zamowienia, id_produktu),
    CONSTRAINT fk_zamowienie
        FOREIGN KEY (id_zamowienia) REFERENCES zamowienia(id_zamowienia),
    CONSTRAINT fk_produkt
        FOREIGN KEY (id_produktu) REFERENCES produkty(id_produktu)
);

```

Wariant SQLite został dostosowany do prostszego modelu typów danych oferowanego przez ten silnik. Typ SERIAL zastąpiono konstrukcją INTEGER PRIMARY KEY AUTOINCREMENT, pola tekstowe oraz daty zapisano jako TEXT, natomiast wartości liczbowe o charakterze ciągłym opisano typem FLOAT.

```

-- Wariant SQLite
CREATE TABLE kategorie (
    id_kategorii INTEGER PRIMARY KEY AUTOINCREMENT,
    nazwa TEXT NOT NULL
);

CREATE TABLE klienci (
    id_klienta INTEGER PRIMARY KEY AUTOINCREMENT,
    imie TEXT NOT NULL,
    nazwisko TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    telefon TEXT,
    ulica TEXT,
    miasto TEXT,
    kod_pocztowy TEXT
);

CREATE TABLE produkty (
    id_produktu INTEGER PRIMARY KEY AUTOINCREMENT,
    id_kategorii INTEGER NOT NULL,
    nazwa TEXT NOT NULL,
    cena_bazowa FLOAT NOT NULL,
    FOREIGN KEY (id_kategorii) REFERENCES kategorie(id_kategorii)
);

CREATE TABLE zamowienia (
    id_zamowienia INTEGER PRIMARY KEY AUTOINCREMENT,
    id_klienta INTEGER NOT NULL,
    data_zlozenia TEXT NOT NULL,
    status TEXT NOT NULL,
    FOREIGN KEY (id_klienta) REFERENCES klienci(id_klienta)
);

CREATE TABLE szczegoly_zamowienia (

```

(continues on next page)

(continued from previous page)

```

id_zamowienia INTEGER NOT NULL,
id_produktu INTEGER NOT NULL,
ilosc INTEGER NOT NULL CHECK (ilosc > 0),
cena_historyczna FLOAT NOT NULL,
PRIMARY KEY (id_zamowienia, id_produktu),
FOREIGN KEY (id_zamowienia) REFERENCES zamowienia(id_zamowienia),
FOREIGN KEY (id_produktu) REFERENCES produkty(id_produktu)
);

```

4.2 Wybór mechanizmu wprowadzania danych

W celu zainicjowania bazy danymi testowymi zdecydowano się na wykorzystanie klasycznych, pojedynczych instrukcji INSERT w języku SQL. Wybór tego bezpośredniego podejścia podyktowany był niewielkim rozmiarem przygotowanego zbioru danych, który służył jedynie do weryfikacji poprawności struktury logicznej i więzów integralności w środowiskach PostgreSQL oraz SQLite. Wykonanie zwykłych skryptów SQL pozwoliło na szybkie zasilenie tabel i natychmiastowe przetestowanie relacji.

Należy jednak wyraźnie zaznaczyć, że w docelowych aplikacjach produkcyjnych lub przy pracy z większymi wolumenami danych, ręczne przygotowywanie instrukcji INSERT jest skrajnie nieefektywne. W takich scenariuszach optymalną praktyką jest agregacja danych wejściowych w ustrukturyzowanych plikach, takich jak JSON lub CSV. Pozwala to na pełną automatyzację za pomocą dedykowanych skryptów (np. w języku Python) oraz wykorzystanie mechanizmów wstawiania wsadowego, np. funkcji `executemany()` z biblioteki `sqlite3` czy narzędzi ORM pokroju `SQLAlchemy`. Przesyłanie wielu rekordów w ramach jednego wywołania sieciowego drastycznie ogranicza narzut komunikacyjny, pozwala na lepsze zarządzanie transakcjami serwera bazy danych i znacząco poprawia ogólną wydajność.

4.3 Komentarz do procesu wprowadzania danych

Proces zasilania bazy danych musiał uwzględniać zależności wynikające z więzów integralności referencyjnej. Z tego względu kolejność wprowadzania danych za pomocą instrukcji INSERT nie mogła być przypadkowa i musiała ściśle odzwierciedlać strukturę relacyjną modelu.

Kolejność wprowadzania danych była następująca:

1. W pierwszej kolejności zaimportowano dane do tabel niezależnych, czyli **kategorie** oraz **klienci**.
2. Następnie zasilono tabelę **produkty**. Operacja ta była możliwa dopiero po wcześniejszym utworzeniu kategorii, ponieważ każdy produkt musi być przypisany do istniejącej encji nadrzędnej.
3. W kolejnym kroku wprowadzono rekordy do tabeli **zamowienia**, przypisując je do istniejących w bazie klientów.
4. Na końcu uzupełniono tabelę **szczegoly_zamowienia**, która pełni funkcję encji asocjacyjnej i łączy identyfikatory zamówień oraz produktów. Wymaga to wcześniejszego istnienia obu powiązanych rekordów.

Takie uporządkowanie procesu importu danych wynika bezpośrednio z konstrukcji schematu relacyjnego i stanowi warunek poprawnego zachowania integralności bazy. W praktyce oznacza to, że dane nadrzędne muszą zostać zapisane przed danymi zależnymi.

ZAPYTANIA DO BAZY DANYCH

Autorzy

Norbert Antonovitch, Michał Bednarczyk, Jan Balazs de Borbatviz

5.1 Ścieżki do wykonanych zapytań na naszej bazie:

Zapytania SQLITE

student02: /lab6

Zapytania PostgreSQL

student01:/Laboratorium7/List6/CLABH01.ipynb

5.2 Funkcje zapytań dla PostgreSQL

Moduł zawierający zapytania analityczne dla bazy PostgreSQL.

`zapytania_postgresql.pobierz_wizytowki_klientow(conn)`

1. Selekcja danych i funkcje wierszowe. Generuje ustandaryzowane wizytówki klientów na potrzeby wysyłek pocztowych.

Zapytanie SQL:

```
SELECT UPPER(imie || ' ' || nazwisko) AS klient_pelna_nazwa,  
       COALESCE(telefon, 'BRAK KONTAKTU') AS telefon_kontaktowy,  
       kod_pocztowy || ' ' || miasto AS adres_wysylkowy,  
       LENGTH(email) AS dlugosc_emaila  
FROM klienci  
ORDER BY nazwisko;
```

Parameters

conn – Obiekt aktywnego połączenia z bazą danych.

Returns

Ramka danych `pandas.DataFrame`.

`zapytania_postgresql.oblicz_statystyki_sprzedazy(conn)`

2. Funkcje agregujące. Oblicza sumaryczne statystyki dla każdego złożonego zamówienia.

Zapytanie SQL:

```
SELECT id_zamowienia,  
       COUNT(id_produktu) AS liczba_unikalnych_produkow,  
       SUM(ilosc) AS laczna_liczba_sztuk,  
       SUM(ilosc * cena_historyczna) AS calkowita_wartosc_zamowienia  
FROM szczegoly_zamowienia  
GROUP BY id_zamowienia  
ORDER BY calkowita_wartosc_zamowienia DESC;
```

Parameters

conn – Obiekt aktywnego połączenia z bazą danych.

Returns

Ramka danych `pandas.DataFrame`.

`zapytania_postgresql.pobierz_pelny_paragon(conn, id_zamowienia)`

3. Połączenia - złączenia. Generuje szczegółowy rachunek (paragon) dla wskazanego zamówienia.

Zapytanie SQL:

```
SELECT z.id_zamowienia, z.data_zlozenia,
       k.imie || ' ' || k.nazwisko AS kupujacy,
       p.nazwa AS nazwa_produkту, sz.ilosc, sz.cena_historyczna
FROM zamowienia z
JOIN klienci k ON z.id_klienta = k.id_klienta
JOIN szczegoly_zamowienia sz ON z.id_zamowienia = sz.id_zamowienia
JOIN produkty p ON sz.id_produkту = p.id_produkту
WHERE z.id_zamowienia = %(id_zam)s;
```

Parameters

- **conn** – Obiekt aktywnego połączenia z bazą danych.
- **id_zamowienia** – Identyfikator zamówienia.

Returns

Ramka danych `pandas.DataFrame`.

`zapytania_postgresql.znajdz_produkty_bez_zamowien(conn)`

4. Operatory zbiorowe. Identyfikuje “martwy” asortyment, czyli produkty, które nigdy nie zostały kupione.

Zapytanie SQL:

```
SELECT id_produkту, nazwa FROM produkty
EXCEPT
SELECT p.id_produkту, p.nazwa
FROM produkty p
JOIN szczegoly_zamowienia sz ON p.id_produkту = sz.id_produkту;
```

Parameters

conn – Obiekt aktywnego połączenia z bazą danych.

Returns

Ramka danych `pandas.DataFrame`.

`zapytania_postgresql.znajdz_asortyment_premium(conn)`

5. Podzapytania. Wyszukuje produkty sklasyfikowane jako asortyment premium.

Zapytanie SQL:

```
SELECT nazwa, cena_bazowa FROM produkty
WHERE cena_bazowa > (
    SELECT AVG(cena_bazowa) FROM produkty
)
ORDER BY cena_bazowa DESC;
```

Parameters

conn – Obiekt aktywnego połączenia z bazą danych.

Returns

Ramka danych `pandas.DataFrame`.

5.3 Funkcje zapytań dla SQLITE

Moduł zawierający zapytania analityczne dla bazy SQLite.

`zapytania_sqlite.raport_domen_pocztowych(conn)`

1. Wyciąganie domeny z maila za pomocą SUBSTR i INSTR.

Zapytanie SQL:

```
SELECT SUBSTR(email, INSTR(email, '@') + 1) AS "Domena_pocztowa",
       COUNT(id_klienta) AS "Liczba_klientow"
FROM klienci
GROUP BY "Domena_pocztowa"
ORDER BY "Liczba_klientow" DESC;
```

Parameters

conn – Obiekt aktywnego połączenia z bazą danych.

Returns

Ramka danych pandas.DataFrame.

`zapytania_sqlite.zawartosc_zamowien_w_linii(conn)`

2. Sklejanie tekstów z wielu wierszy za pomocą GROUP_CONCAT.

Zapytanie SQL:

```
SELECT z.id_zamowienia AS "ID_Zamowienia",
       k.imie || ' ' || k.nazwisko AS "Klient",
       GROUP_CONCAT(p.nazwa, ', ') AS "Lista_produkow"
FROM zamowienia z
JOIN klienci k ON z.id_klienta = k.id_klienta
JOIN szczegoly_zamowienia sz ON z.id_zamowienia = sz.id_zamowienia
JOIN produkty p ON sz.id_produkta = p.id_produkta
GROUP BY z.id_zamowienia;
```

Parameters

conn – Obiekt aktywnego połączenia z bazą danych.

Returns

Ramka danych pandas.DataFrame.

`zapytania_sqlite.terminy_platnosci(conn)`

3. Modyfikacja daty za pomocą wbudowanej funkcji datetime().

Zapytanie SQL:

```
SELECT id_zamowienia AS "ID_Zam",
       data_zlozenia AS "Data_Zamowienia",
       datetime(data_zlozenia, '+7 days') AS "Termin_Platnosci",
       status AS "Status"
FROM zamowienia;
```

Parameters

conn – Obiekt aktywnego połączenia z bazą danych.

Returns

Ramka danych pandas.DataFrame.

`zapytania_sqlite.cennik_opisowy(conn)`

4. Rzutowanie typów (CAST FLOAT -> TEXT) przy łączeniu stringów.

Zapytanie SQL:

```
SELECT nazwa AS "Produkt",
       'Kosztuje: ' || CAST(cena_bazowa AS TEXT) || ' PLN' AS "Cena_Opisowa"
FROM produkty
ORDER BY cena_bazowa DESC;
```

Parameters

conn – Obiekt aktywnego połączenia z bazą danych.

Returns

Ramka danych pandas.DataFrame.

`zapytania_sqlite.czas_od_zamowienia(conn)`

5. Obliczanie różnicy dat za pomocą SQLite'owego julianday().

Zapytanie SQL:

```
SELECT z.id_zamowienia AS "ID_Zamowienia",
       k.imie || ' ' || k.nazwisko AS "Klient",
       z.data_zlozenia AS "Data_Zamowienia",
       CAST(julianday('now') - julianday(z.data_zlozenia) AS INTEGER) AS "Dni_Od_
↪Zlozenia"
FROM zamowienia z
JOIN klienci k ON z.id_klienta = k.id_klienta
ORDER BY "Dni_Od_Zlozenia" DESC;
```

Parameters

conn – Obiekt aktywnego połączenia z bazą danych.

Returns

Ramka danych pandas.DataFrame.